UNIT 3

Algorithms & Programming

100-Question Priority-Starred Master Set

- Questions 1–90: Multiple Choice (Full Priority Ratings & Distractor Breakdowns)
- Questions 91–100: Create Performance Task Style Written Responses (9 Points Each)
- Complete Answer Key and Detailed Worked Solutions at the End

Examination Structure

This paper contains 100 continuously numbered questions designed for complete mastery of AP Computer Science Principles Unit 3: Algorithms and Programming (Big Idea 3: AAP, weighted 30%–35% of the AP Exam - the highest weighted unit). All answers, step-by-step worked solutions, and 9-point scoring rubrics appear after Question 100.

★ Priority Rating System

★★★★★ **Very Important**	High-frequency exam concepts: Variable swapping mechanics, 1-indexed list semantics, Modulo operator arithmetic, Compound Boolean expressions (De Morgan's Laws), Robot Grid path navigation & obstacle avoidance, Linear vs Binary search efficiency, Procedural abstraction parameters & return values, Reasonable vs Unreasonable time (polynomial vs exponential), and Undecidable problems (The Halting Problem).
★★★★ **High Priority**	Core computational standards: Nested conditionals decision trees, REPEAT UNTIL termination conditions, In-place list clamping, Finding min/max with data-driven initializers, Nested iteration matrix counting, String concatenation & 1-based substrings, RANDOM inclusive simulation bounds, Heuristics for intractable problems, and Avoiding concurrent modification bugs during list deletion.
★★★ **Mandatory**	Essential computing mechanics: Arithmetic precedence, Sequential IF overwrites, Off-by-one boundary checks, Boolean flag search loops, Local vs Global variable scope, and Parallel list indexing.

Topic Coverage (Big Idea 3: AAP-1 through AAP-4)

3.1–3.4 Variables, Expressions & Modulo (Assignment, Swapping, Modulo, Strings)

3.5–3.7 Boolean Logic & Conditionals (AND/OR/NOT, De Morgan, Nested IF-ELSE)

3.8–3.9 Iteration & Accumulation (REPEAT n TIMES, REPEAT UNTIL, Loop Bounds)

Robot Grid Navigation & Obstacles (MOVE_FORWARD, ROTATE_RIGHT/LEFT, CAN_MOVE)

3.10 Lists & Data Abstraction (1-Indexed, LENGTH, APPEND, INSERT, REMOVE)

3.10b Advanced List Traversal (FOR EACH, Min/Max, Filtering, In-Place Updates)

3.12–3.14 Procedures & Abstraction (Parameters, RETURN, Scope, Modularity)

3.11 & 3.15 Search & Simulation (Linear vs. Binary Search, RANDOM, Simulations)

3.16–3.17 Efficiency & Decidability (Reasonable/Unreasonable Time, Heuristics, Halting)

AP Exam Pro Tip: Remember: In the official AP CSP Reference Sheet, **lists and strings are strictly 1-indexed** (`list[1]` is the first element)! In robot grids, `ROTATE_RIGHT()` and `ROTATE_LEFT()` rotate 90° in place without changing position. In binary search, the list **must be sorted** beforehand. And for undecidable problems, remember Alan Turing's Halting Problem proves that no algorithm can ever be built that correctly determines if any arbitrary program halts!

- **Questions 1–90:** Multiple choice, four options (A–D), exactly one correct answer.
- **Questions 91–100:** Create Performance Task Written Response scenarios, 9 points each, with full rubrics.
- **Difficulty Mix:** Authentic Medium AP Exam Level ❖ Rigorously 3-Pass Verified.
- **Answer Section:** Begins after Question 100 — master answer key, detailed MCQ solutions with distractor failure analysis, and model FRQ solutions with official point allocations.

Question 01

★★★★★ VERY IMPORTANT

Variable Assignment & Sequential Execution

Consider the following code segment:

```
first <- 15
second <- 30
first <- second
second <- first
DISPLAY(first)
DISPLAY(second)
```

What is displayed after the code segment executes?

(A) 30 30

(B) 15 15

(C) 15 30

(D) 30 15

Question 02

★★★★★ VERY IMPORTANT

Variable Swapping Algorithm

A programmer wants to swap the values stored in variables `x` and `y`. For example, if `x` is initially 7 and `y` is initially 12, after the swap `x` should store 12 and `y` should store 7.

Which of the following code segments correctly swaps the values of `x` and `y`?

(A)
```
x <- temp
temp <- y
y <- x
```

(B)
```
temp <- y
y <- x
x <- y
```

(C)
```
temp <- x
x <- y
y <- temp
```

(D)
```
x <- y
temp <- x
y <- temp
```

Question 03

★★★★ HIGH PRIORITY

Modulo Operator & Clock Arithmetic

In AP CSP pseudocode, the expression `a MOD b` evaluates to the remainder when `a` is divided by `b`. Consider the following code segment intended to calculate the time on a 12-hour clock 15 hours after `currentHour`:

```
currentHour <- 9
elapsedHours <- 15
newHour <- (currentHour + elapsedHours) MOD 12
IF (newHour = 0)
{
    newHour <- 12
}
DISPLAY(newHour)
```

What value is displayed?

(A) 0

(B) 6

(C) 24

(D) 12

Question 04

★★★★ HIGH PRIORITY

Arithmetic Operator Precedence & Evaluation

Consider the following code segment:

```
p <- 4
q <- 6
r <- 2
result <- p + q * r - 14 MOD 4
DISPLAY(result)
```

What value is displayed?

(A) 16

(B) 18

(C) 12

(D) 14

Question 05

★★★★ HIGH PRIORITY

Sequential Accumulator Updates

Consider the following code segment:

```
total <- 20
factor <- 3
total <- total * factor
factor <- factor + 2
total <- total - factor
DISPLAY(total)
```

What value is displayed after the code segment executes?

(A) 55

(B) 57

(C) 60

(D) 53

Question 06

★★★★★ VERY IMPORTANT

Divisibility Testing Using Modulo

A programmer wants to write a Boolean condition that evaluates to `true` if and only if a positive integer variable `n` is divisible by both 4 and 6, and evaluates to `false` otherwise.

Which of the following expressions correctly implements this requirement?

(A) (n MOD 4 = 0) OR (n MOD 6 = 0)

(B) (n MOD 24 = 0)

(C) (n MOD 4 = 0) AND (n MOD 6 = 0)

(D) (n MOD 10 = 0)

Question 07

★★★★ HIGH PRIORITY

String Concatenation & Substring Operations

In AP CSP pseudocode, strings can be concatenated using `concat(str1, str2)` or the `+` operator. The procedure `substring(str, start, len)` returns a string of length `len` starting at 1-based index `start`.

Consider the following code segment:

```
prefix <- "data"
suffix <- "base"
fullWord <- concat(prefix, suffix)
part <- substring(fullWord, 3, 4)
DISPLAY(part)
```

What value is displayed?

(A) atab

(B) data

(C) taba

(D) base

Question 08

★★★★ HIGH PRIORITY

Equivalent Mathematical Expressions in Pseudocode

A student needs to compute the average of three quiz scores stored in variables `q1`, `q2`, and `q3` and then add a 5-point curve to the average.

Which of the following code expressions correctly calculates this curved average?

(A) (q1 + q2 + q3) / (3 + 5)

(B) (q1 + q2 + q3) / 3 + 5

(C) (q1 + q2 + q3 + 5) / 3

(D) q1 + q2 + q3 / 3 + 5

Question 09

★★★★★ VERY IMPORTANT

Instruction Sequencing & Side Effects

Consider the following initial variables:

```
x <- 10
y <- 20
```

A programmer compares two different code segments:

Segment 1:

```
x <- x + y
y <- x - y
```

Segment 2:

```
y <- x - y
x <- x + y
```

What are the resulting values of `x` and `y` in Segment 1 and Segment 2, respectively?

(A) Segment 1: x = 30, y = -10; Segment 2: x = 10, y = -10

(B) Segment 1: x = 30, y = 10; Segment 2: x = 30, y = 10

(C) Segment 1: x = 10, y = 30; Segment 2: x = -10, y = 0

(D) Segment 1: x = 30, y = 10; Segment 2: x = 0, y = -10

Question 10

★★★★ HIGH PRIORITY

Digit Extraction Using Modulo and Division

A programmer wants to extract the tens digit of a positive two-digit integer stored in variable `num` (for example, if `num` is 47, the tens digit is 4). In AP CSP pseudocode, integer division discards any fractional part.

Which of the following code expressions correctly extracts the tens digit?

(A) num - 10

(B) (num MOD 10) / 10

(C) num MOD 10

(D) num / 10

Question 11

★★★★★ VERY IMPORTANT

Compound Boolean Expressions with AND, OR, NOT

Consider the following variable assignments:

```
isWeekend <- true
isHoliday <- false
hasHomework <- true
canPlayVideoGames <- (isWeekend OR isHoliday) AND (NOT hasHomework)
DISPLAY(canPlayVideoGames)
```

What value is displayed?

(A) true

(B) null

(C) false

(D) error

Question 12

★★★★★ VERY IMPORTANT

De Morgan's Laws & Condition Simplification

A programmer wants to simplify the following conditional expression:

```
NOT (score > 85 OR attendance < 90)
```

Which of the following expressions is logically equivalent for all integer values of `score` and `attendance`?

(A) (score <= 85) OR (attendance >= 90)

(B) NOT (score > 85) OR NOT (attendance < 90)

(C) (score <= 85) AND (attendance >= 90)

(D) (score < 85) AND (attendance > 90)

Question 13

★★★★ HIGH PRIORITY

Truth Table Evaluation

Consider two Boolean variables, `p` and `q`. Which of the following expressions evaluates to `true` if and only if `p` is `true` and `q` is `false`?

(A) p AND (NOT q)

(B) (NOT p) AND q

(C) NOT (p AND q)

(D) p OR (NOT q)

Question 14

★★★★ HIGH PRIORITY

Operator Precedence in Boolean Expressions

Consider the following variable assignments:

```
a <- false
b <- true
c <- true
result <- a OR b AND c
DISPLAY(result)
```

In standard programming and AP CSP logic conventions, `AND` has higher precedence than `OR` unless overridden by parentheses. What value is displayed?

(A) true

(B) undefined

(C) null

(D) false

Question 15

★★★★★ VERY IMPORTANT

Boundary Value Analysis in Relational Expressions

A school discount program applies to students who are between 14 and 18 years old, inclusive. A developer writes two different code conditions to test if a student of age `studentAge` qualifies:

Condition 1: `(studentAge >= 14) AND (studentAge <= 18)`
Condition 2: `(studentAge > 13) AND (studentAge < 19)`

Assuming `studentAge` is always an integer, which of the following best describes the behavior of these two conditions?

(A) Condition 1 evaluates to true for non-integer inputs, while Condition 2 does not.

(B) Condition 2 includes 13 and 19, but Condition 1 excludes them.

(C) Condition 1 includes 14 and 18, but Condition 2 excludes them.

(D) Condition 1 and Condition 2 evaluate to the exact same Boolean result for all possible integer values of studentAge.

Question 16

★★★★ HIGH PRIORITY

Double Negation & Boolean Simplification

Consider the following Boolean expression:

```
NOT (NOT p AND NOT q)
```

Which of the following is logically equivalent to this expression for all Boolean values of `p` and `q`?

(A) p AND q

(B) NOT (p OR q)

(C) NOT p OR NOT q

(D) p OR q

Question 17

★★★★★ VERY IMPORTANT

Range Exclusion vs Inclusion Logic

A quality control sensor in a manufacturing plant flags an item as defective if its temperature in Celsius is either strictly below 15°C or strictly above 85°C.

Which of the following expressions evaluates to `true` when the temperature `temp` is flagged as defective?

(A) (temp < 15) AND (temp > 85)

(B) (temp <= 15) OR (temp >= 85)

(C) (temp < 15) OR (temp > 85)

(D) NOT ((temp < 15) OR (temp > 85))

Question 18

★★★★ HIGH PRIORITY

Boolean Flags in Search Logic

Consider the following code segment intended to check if the target value 42 exists in a small collection:

```
found <- false
val1 <- 18
val2 <- 42
val3 <- 99

IF (val1 = 42)
{
    found <- true
}
IF (val2 = 42)
{
    found <- true
}
IF (val3 = 42)
{
    found <- true
}
DISPLAY(found)
```

What value is displayed?

(A) true

(B) false

(C) 42

(D) error

Question 19

★★★★ HIGH PRIORITY

Complex Expression Evaluation with Multiple Variables

Consider the following assignments:

```
x <- 5
y <- 12
z <- 20
condition <- (x * 2 > y) OR (z / 2 = 10 AND y < x)
DISPLAY(condition)
```

What value is displayed?

(A) false

(B) 10

(C) true

(D) null

Question 20

★★★★★ VERY IMPORTANT

Logical Contradiction Bug in Conditionals

A novice programmer wants to alert laboratory technicians whenever a refrigerator temperature `temp` is outside the safe range of 2°C to 8°C. The programmer writes:

```
IF (temp < 2 AND temp > 8)
{
    DISPLAY("Warning: Unsafe Temperature!")
}
```

Why does the warning message never display, regardless of the value of `temp`?

(A) The conditional statement requires an ELSE clause in order to evaluate.

(B) No single number can be simultaneously less than 2 and greater than 8, making the AND condition a logical contradiction that always evaluates to false.

(C) The DISPLAY procedure cannot output string literals.

(D) The relational operators `<` and `>` cannot be used with integer variables.

Question 21

★★★★★ VERY IMPORTANT

Sequential IF Statements vs. Mutually Exclusive Branches

Consider the following code segment intended to assign a letter grade based on a numeric score:

```
score <- 95
grade <- "F"

IF (score >= 90)
{
    grade <- "A"
}
IF (score >= 80)
{
    grade <- "B"
}
IF (score >= 70)
{
    grade <- "C"
}
IF (score >= 60)
{
    grade <- "D"
}
DISPLAY(grade)
```

What value is displayed?

(A) "F"

(B) "B"

(C) "D"

(D) "A"

Question 22

★★★★ HIGH PRIORITY

Nested Conditionals Decision Tree

Consider the following nested conditional code segment:

```
age <- 16
hasPermit <- true
hoursDriven <- 45
status <- ""

IF (age >= 16)
{
    IF (hasPermit = true)
    {
        IF (hoursDriven >= 50)
        {
            status <- "Eligible for License"
        }
        ELSE
        {
            status <- "Needs More Practice"
        }
    }
    ELSE
    {
        status <- "Needs Permit"
    }
}
ELSE
{
    status <- "Too Young"
}
DISPLAY(status)
```

What value is displayed?

(A) "Eligible for License"

(B) "Needs Permit"

(C) "Too Young"

(D) "Needs More Practice"

Question 23

★★★★★ VERY IMPORTANT

Equivalence of Nested IFs and Compound AND

A software engineer compares two implementations designed to grant administrative system access:

Implementation 1:

```
access <- false
IF (isEmployee = true)
{
    IF (hasBadge = true)
    {
        access <- true
    }
}
```

Implementation 2:

```
access <- false
IF (isEmployee = true AND hasBadge = true)
{
    access <- true
}
```

Which of the following statements correctly compares the two implementations?

(A) Implementation 1 and Implementation 2 produce the exact same value for access for all combinations of Boolean inputs.

(B) Implementation 1 grants access if isEmployee is true, even if hasBadge is false.

(C) Implementation 2 grants access when only one of the two conditions is true.

(D) Implementation 1 requires more Boolean logic operations than Implementation 2.

Question 25

★★★★ HIGH PRIORITY

Tiered Pricing & Nested Discounts

A movie theater determines ticket price using the following algorithm:

```
basePrice <- 12
age <- 68
isMatinee <- true

IF (age >= 65)
{
    basePrice <- 8
}
ELSE
{
    IF (age <= 12)
    {
        basePrice <- 6
    }
}

IF (isMatinee = true)
{
    basePrice <- basePrice - 2
}
DISPLAY(basePrice)
```

What value is displayed?

(A) 4

(B) 6

(C) 8

(D) 10

Question 26

★★★★★ VERY IMPORTANT

Detecting Dead/Unreachable Code

Consider the following code segment where `val` is an unknown integer:

```
IF (val > 50)
{
    IF (val < 20)
    {
        DISPLAY("Target detected")
    }
    ELSE
    {
        DISPLAY("High range")
    }
}
ELSE
{
    DISPLAY("Standard range")
}
```

Which of the following statements is true regarding this code segment?

(A) Every integer value of val will display "Target detected".

(B) The message "Standard range" will never be displayed.

(C) The message "Target detected" will never be displayed for any integer value of val.

(D) The message "High range" will never be displayed.

Question 27

★★★★ HIGH PRIORITY

Cumulative Score & Independent Conditionals

A driver's penalty points are calculated using the following code segment:

```
points <- 0
speed <- 72
speedLimit <- 55
isSchoolZone <- false
```

Question 28

★★★★ HIGH PRIORITY

Converting Nested Conditionals to Single Expressions

Consider the following nested conditional statement:

```
canEnroll <- false
IF (hasPrerequisite = true)
{
    IF (classFull = false)
    {
        canEnroll <- true
    }
}
```

Which of the following single conditional statements has the exact same functionality?

(A) IF (NOT hasPrerequisite OR classFull)
{
canEnroll <- true
}

(B) IF (hasPrerequisite = true OR NOT classFull)
{
canEnroll <- true
}

(C) IF (hasPrerequisite = true AND classFull)
{
canEnroll <- true
}

(D) IF (hasPrerequisite = true AND NOT classFull)
{
canEnroll <- true
}

Question 29

★★★★★ VERY IMPORTANT

Shipping Tier Boundary Error Debugging

An online store calculates shipping fees according to the following policy:
• Purchases under $50 pay $10 shipping.
• Purchases from $50 up to $100 pay $5 shipping.
• Purchases over $100 receive free shipping ($0).

A developer writes:

```
IF (orderTotal < 50)
{
    shipping <- 10
}
ELSE
{
    IF (orderTotal < 100)
    {
        shipping <- 5
    }
    ELSE
    {
        shipping <- 0
    }
}
```

For which of the following order amounts does this algorithm calculate an INCORRECT shipping fee according to the stated store policy?

(A) 100

(B) 49.99

(C) 50

(D) 100.01

Question 30

★★★★ HIGH PRIORITY

Nested Conditional State Machine Trace

Consider the following code segment tracking a drone's altitude:

Question 31

★★★★★ VERY IMPORTANT

REPEAT n TIMES Accumulator Trace

Consider the following code segment:

```
count <- 0
sum <- 0

REPEAT 4 TIMES
{
    count <- count + 1
    sum <- sum + count
}
DISPLAY(sum)
```

What value is displayed?

(A) 4

(B) 6

(C) 10

(D) 16

Question 32

★★★★★ VERY IMPORTANT

REPEAT UNTIL Termination Condition

In AP CSP pseudocode, `REPEAT UNTIL (condition)` repeatedly executes the code block until the Boolean condition evaluates to `true`. If the condition is initially `true`, the block does not execute at all.

Consider the following code segment:

```
n <- 1
REPEAT UNTIL (n >= 8)
{
    n <- n * 2
}
DISPLAY(n)
```

What value is displayed?

(A) 7

(B) 4

(C) 8

(D) 16

Question 33

★★★★ HIGH PRIORITY

Loop Algorithm Equivalence

A programmer wants to calculate the sum of all positive even integers from 2 to 10, inclusive (2 + 4 + 6 + 8 + 10 = 30).

Which of the following code segments correctly computes this sum?

(A)
```
sum <- 0
i <- 2
REPEAT 5 TIMES
{
sum <- sum + i
i <- i + 2
}
```

(B)
```
sum <- 0
i <- 2
REPEAT UNTIL (i = 10)
{
sum <- sum + i
i <- i + 2
}
```

(C)
```
sum <- 0
i <- 1
REPEAT 5 TIMES
{
sum <- sum + i * 2
}
```

(D)
```
sum <- 0
```

Question 34

★★★★★ VERY IMPORTANT

Infinite Loop Identification

Consider the following code segment:

```
i <- 1
count <- 0

REPEAT UNTIL (i = 10)
{
    count <- count + 1
    i <- i + 2
}
DISPLAY(count)
```

What is the result of executing this code segment?

(A) The code segment displays 4.

(B) The code segment terminates with a syntax error.

(C) The code segment displays 5.

(D) The code segment results in an infinite loop because i will skip over 10 and never equal 10.

Question 35

★★★★ HIGH PRIORITY

Nested Iteration Execution Count

Consider the following code segment:

```
totalStars <- 0

REPEAT 3 TIMES
{
    REPEAT 4 TIMES
    {
        totalStars <- totalStars + 2
    }
}
DISPLAY(totalStars)
```

What value is displayed?

(A) 12

(B) 24

(C) 14

(D) 48

Question 36

★★★★ HIGH PRIORITY

Loop Iteration Counting

Consider the following code segment:

```
val <- 25
steps <- 0

REPEAT UNTIL (val < 5)
{
    val <- val - 6
    steps <- steps + 1
}
DISPLAY(steps)
```

What value is displayed?

(A) 5

(B) 3

(C) 4

(D) 25

Question 37

★★★★★ VERY IMPORTANT

Off-by-One Boundary Error in Iteration

A student wants to print the numbers 1 through 5, inclusive (1, 2, 3, 4, 5). The student writes the following code segment:

```
num <- 1
REPEAT UNTIL (num = 5)
{
    DISPLAY(num)
    num <- num + 1
}
```

What is the actual output of this code segment?

(A) It displays 1, 2, 3, 4 (stops before displaying 5).

(B) It displays 1, 2, 3, 4, 5.

(C) It results in an infinite loop.

(D) It displays 1, 2, 3, 4, 5, 6.

Question 38

★★★★ HIGH PRIORITY

Selection Nested Within Iteration

Consider the following code segment:

```
evenCount <- 0
i <- 1

REPEAT 6 TIMES
{
    IF (i MOD 2 = 0)
    {
        evenCount <- evenCount + 1
    }
    i <- i + 1
}
DISPLAY(evenCount)
```

What value is displayed?

(A) 3

(B) 4

(C) 2

(D) 6

Question 39

★★★★★ VERY IMPORTANT

Logarithmic Repeated Division Trace

Consider the following code segment:

```
num <- 64
steps <- 0

REPEAT UNTIL (num <= 1)
{
    num <- num / 2
    steps <- steps + 1
}
DISPLAY(steps)
```

What value is displayed?

(A) 7

(B) 6

(C) 64

(D) 5

Question 40

★★★★ HIGH PRIORITY

Trace Table with Interdependent Loop Updates

Consider the following code segment:

```
a <- 2
b <- 5

REPEAT 3 TIMES
{
    a <- a + b
    b <- a - b
}
DISPLAY(a)
DISPLAY(b)
```

What is displayed after the code segment executes?

(A) 7 2

(B) 25 16

(C) 9 7

(D) 16 9

Question 41

★★★★★ VERY IMPORTANT

Robot Grid Sequential Navigation

A robot is situated in the bottom-left corner (row 4, column 1) of a 4x4 grid of squares, facing North (▲). The grey goal square is located in row 2, column 3.

```
+---+---+---+---+
| . | . | . | . |  Row 1
+---+---+---+---+
| . | . |[G]| . |  Row 2  ([G] = Grey Goal)
+---+---+---+---+
| . | . | . | . |  Row 3
+---+---+---+---+
| ▲ | . | . | . |  Row 4  (▲ = Robot facing North)
+---+---+---+---+
 Col 1 Col 2 Col 3 Col 4
```

Which of the following code segments will successfully move the robot to the grey goal square [G]?

(A)
ROTATE_RIGHT()
MOVE_FORWARD()
MOVE_FORWARD()
ROTATE_LEFT()
MOVE_FORWARD()

(B)
MOVE_FORWARD()
ROTATE_RIGHT()
MOVE_FORWARD()
MOVE_FORWARD()

(C)
MOVE_FORWARD()
MOVE_FORWARD()
ROTATE_RIGHT()
MOVE_FORWARD()
MOVE_FORWARD()

(D)
MOVE_FORWARD()
MOVE_FORWARD()
MOVE_FORWARD()
ROTATE_RIGHT()
MOVE_FORWARD()

Question 42

★★★★ HIGH PRIORITY

Robot Iteration with REPEAT Loops

A robot begins at (Row 5, Col 1) facing North (▲) on a 5x5 grid. The programmer writes the following code segment:

```
REPEAT 3 TIMES
{
    MOVE_FORWARD()
}
ROTATE_RIGHT()
```

Question 43

★★★★★ VERY IMPORTANT

Robot Obstacle Avoidance Around Black Squares

Consider the following 4x4 grid where a black square [X] represents an impassable obstacle at (Row 2, Col 2). The robot begins at (Row 3, Col 2) facing North (▲). The grey goal [G] is directly behind the obstacle at (Row 1, Col 2).

```
+---+---+---+---+
| . |[G]| . | . |  Row 1  ([G] = Goal at R1, C2)
+---+---+---+---+
| . |[X]| . | . |  Row 2  ([X] = Black obstacle at R2, C2)
+---+---+---+---+
| . | ▲ | . | . |  Row 3  (▲ = Robot at R3, C2 facing North)
+---+---+---+---+
| . | . | . | . |  Row 4
+---+---+---+---+
 Col 1 Col 2 Col 3 Col 4
```

If the robot cannot move into an obstacle square, which of the following code segments safely navigates the robot to the goal square [G] without crashing?

(A)
```
ROTATE_RIGHT()
MOVE_FORWARD()
ROTATE_LEFT()
MOVE_FORWARD()
MOVE_FORWARD()
ROTATE_LEFT()
MOVE_FORWARD()
```

(B)
```
MOVE_FORWARD()
ROTATE_RIGHT()
MOVE_FORWARD()
ROTATE_LEFT()
MOVE_FORWARD()
```

(C)
```
ROTATE_RIGHT()
MOVE_FORWARD()
ROTATE_LEFT()
MOVE_FORWARD()
ROTATE_LEFT()
MOVE_FORWARD()
```

(D)
```
ROTATE_LEFT()
MOVE_FORWARD()
ROTATE_RIGHT()
MOVE_FORWARD()
ROTATE_RIGHT()
MOVE_FORWARD()
```

Question 44

★★★★★ VERY IMPORTANT

CAN_MOVE Condition in Termination Loops

In AP CSP pseudocode, `CAN_MOVE(direction)` evaluates to `true` if the robot can move one square in the specified relative direction (`forward`, `left`, `right`, `backward`) without hitting a grid wall or black obstacle, and `false` otherwise.

Consider the following code segment:

```
REPEAT UNTIL (CAN_MOVE(forward) = false)
{
    MOVE_FORWARD()
}
ROTATE_RIGHT()
MOVE_FORWARD()
```

A robot is placed on an open 5x5 grid at (Row 4, Col 2) facing North (▲) with no obstacles. What will be the final position of the robot?

(A) Row 2, Column 2, facing East

(B) Row 1, Column 3, facing East

(C) Row 1, Column 1, facing West

(D) Row 1, Column 2, facing North

Question 45

★★★★ HIGH PRIORITY

Right-Hand Wall Following Algorithm

Consider the following algorithmic procedure for navigating a maze corridor:

Question 46

★★★★★ VERY IMPORTANT

Algorithm Comparison for Grid Navigation

A robot must travel from the center of a room (Row 3, Col 3) to the bottom-right corner (Row 5, Col 5) on a 5x5 grid. The robot starts facing South (▼).

Consider the following two algorithms:

Algorithm 1:

```
MOVE_FORWARD()
MOVE_FORWARD()
ROTATE_LEFT()
MOVE_FORWARD()
MOVE_FORWARD()
```

Algorithm 2:

```
REPEAT 2 TIMES
{
    MOVE_FORWARD()
    ROTATE_LEFT()
    MOVE_FORWARD()
    ROTATE_RIGHT()
}
```

Which of the following statements is true?

(A) Only Algorithm 2 reaches (Row 5, Col 5); Algorithm 1 ends at (Row 1, Col 5).

(B) Only Algorithm 1 reaches (Row 5, Col 5); Algorithm 2 crashes into a wall.

(C) Neither algorithm reaches (Row 5, Col 5).

(D) Both Algorithm 1 and Algorithm 2 successfully move the robot to (Row 5, Col 5).

Question 47

★★★★ HIGH PRIORITY

Perimeter Traversal Loop Counting

A robot is placed in the top-left corner (Row 1, Col 1) of a 4x4 room facing East (▶). The programmer runs the following code to walk the robot around the outer perimeter of the room:

```
REPEAT 4 TIMES
{
    REPEAT 3 TIMES
    {
        MOVE_FORWARD()
    }
    ROTATE_RIGHT()
}
```

How many total times is the `MOVE_FORWARD()` command executed during the run?

(A) 12

(B) 16

(C) 7

(D) 9

Question 48

★★★★ HIGH PRIORITY

Nested Robot Pattern Trace

A robot begins at (Row 4, Col 1) facing East (▶) on a 4x4 grid. Consider the following code segment:

```
REPEAT 3 TIMES
{
    MOVE_FORWARD()
    ROTATE_LEFT()
    MOVE_FORWARD()
    ROTATE_RIGHT()
}
```

What is the robot's position after the code executes?

(A) Row 1, Column 1

(B) Row 1, Column 4

(C) Row 4, Column 4

(D) Row 2, Column 3

Question 49

★★★★★ VERY IMPORTANT

Robot Orientation Bug Diagnostics

A student writes a script to move a robot from (Row 3, Col 1) facing North to (Row 1, Col 3). The student writes:

```
MOVE_FORWARD()
MOVE_FORWARD()
ROTATE_LEFT()
MOVE_FORWARD()
MOVE_FORWARD()
```

When tested on the grid, the robot attempts to move off the left side of the grid on line 4 and crashes into the outer wall. What was the student's logical error?

(A) The MOVE_FORWARD() command defaults to South if called consecutively.

(B) The student used ROTATE_LEFT() instead of ROTATE_RIGHT(), causing the robot to face West instead of East.

(C) The student should have moved forward 3 times before rotating.

(D) The robot is not permitted to move twice in a row without turning.

Question 50

★★★★★ VERY IMPORTANT

Robot Infinite Oscillation Loop

A robot is placed in an open 4x4 room with no obstacles, facing North. A student writes the following loop:

```
REPEAT UNTIL (CAN_MOVE(forward) = false)
{
    ROTATE_RIGHT()
    ROTATE_LEFT()
}
```

What happens when this code segment executes?

(A) The robot moves to the north wall and stops.

(B) The program terminates immediately with an execution error.

(C) The code results in an infinite loop because the robot oscillates in place without moving, so the wall ahead is never reached.

(D) The robot moves around the four edges of the room.

Question 51

★★★★★ VERY IMPORTANT

1-Based Indexing in College Board Pseudocode

In AP CSP pseudocode, list indices are 1-based (the first element is at index 1). Consider the following code segment:

```
colors <- ["red", "green", "blue", "yellow"]
DISPLAY(colors[1])
DISPLAY(colors[3])
```

What is displayed after the code segment executes?

(A) red green

(B) green blue

(C) red blue

(D) green yellow

Question 52

★★★★★ VERY IMPORTANT

APPEND and LENGTH Operations

Consider the following code segment:

```
scores <- [85, 92, 78]
APPEND(scores, 95)
APPEND(scores, 88)
DISPLAY(LENGTH(scores))
DISPLAY(scores[LENGTH(scores)])
```

What is displayed?

(A) 4 88

(B) 3 78

(C) 5 95

(D) 5 88

Question 53

★★★★★ VERY IMPORTANT

INSERT Operation & Element Shifting

In AP CSP pseudocode, `INSERT(aList, i, value)` inserts `value` at index `i`, shifting the element previously at index `i` and all subsequent elements one position to the right.

Consider the following code segment:

```
letters <- ["A", "B", "D", "E"]
INSERT(letters, 3, "C")
DISPLAY(letters[3])
DISPLAY(letters[4])
```

What is displayed?

(A) C E

(B) B C

(C) D E

(D) C D

Question 54

★★★★★ VERY IMPORTANT

REMOVE Operation & Left Shifting

In AP CSP pseudocode, `REMOVE(aList, i)` removes the element at index `i`, shifting all subsequent elements one position to the left and decreasing the list length by 1.

Consider the following code segment:

```
items <- ["pen", "notebook", "eraser", "ruler"]
REMOVE(items, 2)
DISPLAY(items[2])
DISPLAY(LENGTH(items))
```

What is displayed?

(A) pen 3

(B) eraser 3

(C) notebook 3

(D) eraser 4

Question 55

★★★★ HIGH PRIORITY

Interleaved List Insertion and Deletion

Consider the following code segment:

```
animals <- ["cat", "dog", "bird"]
INSERT(animals, 2, "fish")
REMOVE(animals, 3)
APPEND(animals, "hamster")
DISPLAY(animals[3])
```

What value is displayed?

(A) "hamster"

(B) "dog"

(C) "bird"

(D) "fish"

Question 56

★★★★★ VERY IMPORTANT

Swapping Elements in a List

A programmer wants to swap the first and second elements in a list `nums <- [40, 90, 60]`. After the code executes, `nums` should

Which of the following code segments correctly performs this swap?

(A)
```
nums[1] <- temp
temp <- nums[2]
nums[2] <- nums[1]
```

(B)
```
nums[1] <- nums[2]
nums[2] <- nums[1]
```

(C)
```
temp <- nums[1]
nums[1] <- nums[2]
nums[2] <- temp
```

(D)
```
temp <- nums[2]
nums[2] <- nums[1]
nums[1] <- nums[2]
```

Question 57

★★★★★ VERY IMPORTANT

Managing Program Complexity with Data Abstraction

A student writes a program to track test scores for a class of 30 students. The student considers two design approaches:

- Approach A: Create 30 distinct variables (`score1`, `score2`, ..., `score30`).
- Approach B: Create a single list `scores` with 30 elements.

Which of the following best explains why Approach B represents a superior data abstraction for managing program complexity?

(A) Approach B allows algorithms (such as computing class average or finding the highest score) to be written concisely using loops, and easily accommodates changes in class size without rewriting code.

(B) Approach B eliminates the need for mathematical comparison operators.

(C) Approach B executes faster on hardware because lists bypass computer memory.

(D) Approach B prevents all possible runtime errors and logic errors in the program.

Question 58

★★★★ HIGH PRIORITY

Index Out of Bounds Error Analysis

Consider the following code segment:

```
roster <- ["Alex", "Blake", "Casey"]
DISPLAY(roster[0])
```

In standard AP CSP pseudocode, what occurs when this code segment executes?

(A) It displays "Alex".

(B) It displays an empty string.

(C) It displays "Casey".

(D) An index out-of-bounds error occurs because AP CSP lists are 1-based and 0 is not a valid index.

Question 59

★★★★ HIGH PRIORITY

Conditional Element Modification in Lists

Consider the following code segment:

```
values <- [12, 45, 8, 26]

IF (values[1] < 20)
{
    values[1] <- values[1] * 2
}
IF (values[3] < 10)
{
    values[3] <- values[3] + 10
}
DISPLAY(values[1])
DISPLAY(values[3])
```

What is displayed after the code segment executes?

(A) 24 8

(B) 12 18

(C) 12 8

(D) 24 18

Question 60

★★★★ HIGH PRIORITY

List Assignment and Element Isolation

Consider the following code segment:

```
listA <- [10, 20, 30]
val <- listA[2]
val <- val + 5
listA[2] <- 99
DISPLAY(val)
DISPLAY(listA[2])
```

What is displayed?

(A) 25 25

(B) 99 99

(C) 25 99

(D) 20 99

Question 61

★★★★★ VERY IMPORTANT

FOR EACH List Traversal & Accumulation

Consider the following code segment:

```
numbers <- [4, 8, 12, 16]
total <- 0

FOR EACH item IN numbers
{
    total <- total + item
}
DISPLAY(total)
```

What value is displayed?

(A) 24

(B) 16

(C) 40

(D) 4

Question 62

★★★★★ VERY IMPORTANT

Maximum Element Algorithm & Initializer Bug

A student writes the following algorithm to find the maximum value in a list of integers `numbers`:

```
maxVal <- 0
FOR EACH val IN numbers
{
    IF (val > maxVal)
    {
        maxVal <- val
    }
}
DISPLAY(maxVal)
```

For which of the following lists will this algorithm produce an INCORRECT result?

(A) [10, 25, 5, 18]

(B) [-15, -8, -42, -5]

(C) [-10, 0, 5, -2]

(D) [0, 5, 2, 8]

Question 63

★★★★ HIGH PRIORITY

Finding the Index of the Minimum Element

Consider the following algorithm designed to find the index of the smallest element in a list:

```
scores <- [88, 72, 95, 64, 80]
minIndex <- 1
i <- 2

REPEAT UNTIL (i > LENGTH(scores))
{
    IF (scores[i] < scores[minIndex])
    {
        minIndex <- i
    }
    i <- i + 1
}
DISPLAY(minIndex)
DISPLAY(scores[minIndex])
```

What is displayed after the code segment executes?

(A) 2 72

(B) 4 64

(C) 5 80

(D) 64 4

Question 64

★★★★★ VERY IMPORTANT

List Filtering Algorithm

A programmer wants to filter a list of temperatures `temps` to keep only values that are strictly greater than 32°F, storing the filtered values in a new list `aboveFreezing`.

Which of the following code segments correctly accomplishes this?

(A)
```
aboveFreezing <- []
IF (temps > 32)
{
aboveFreezing <- temps
}
```

(B)
```
aboveFreezing <- temps
FOR EACH t IN aboveFreezing
{
IF (t < 32)
{
REMOVE(aboveFreezing, t)
}
}
```

(C)
```
aboveFreezing <- []
FOR EACH t IN temps
{
IF (t >= 32)
{
APPEND(aboveFreezing, t)
}
}
```

(D)
```
aboveFreezing <- []
FOR EACH t IN temps
{
IF (t > 32)
{
APPEND(aboveFreezing, t)
}
}
```

Question 65

★★★★ HIGH PRIORITY

In-Place List Element Clamping

Consider the following code segment intended to replace all negative numbers in a list with zero:

```
readings <- [14, -3, 22, -9, 0]
i <- 1

REPEAT LENGTH(readings) TIMES
{
    IF (readings[i] < 0)
    {
        readings[i] <- 0
    }
    i <- i + 1
}
```

What is the resulting content of `readings` after the code segment executes?

(A) [0, 0, 0, 0, 0]

(B) [14, -3, 22, -9, 0]

(C) [14, 0, 22, -9, 0]

(D) [14, 0, 22, 0, 0]

Question 66

★★★★ HIGH PRIORITY

Target Element Occurrence Counter

Consider the following code segment:

```
votes <- ["YES", "NO", "YES", "YES", "NO", "ABSTAIN"]
yesCount <- 0

FOR EACH v IN votes
```

Question 67

★★★★★ VERY IMPORTANT

Index-Based Traversal vs. FOR EACH Traversal

> A programmer is deciding whether to traverse a list using an index loop (`REPEAT LENGTH(aList) TIMES` with a counter `i`) or a `FO`

In which of the following situations is an index-based loop REQUIRED instead of a `FOR EACH` loop?

(A) When calculating the sum of all numeric elements in the list.

(B) When the algorithm needs to modify list elements at specific positions in place, or needs to access the adjacent element (such as comparing aList[i] with aList[i + 1]).

(C) When counting how many numbers in the list are greater than 100.

(D) When searching for a target string without needing to report its index position.

Question 68

★★★★ HIGH PRIORITY

Comparing Two Lists for Exact Equality

Consider the following algorithm designed to determine if two lists `list1` and `list2` are identical in length and content:

```
areEqual <- true

IF (LENGTH(list1) ≠ LENGTH(list2))
{
    areEqual <- false
}
ELSE
{
    i <- 1
    REPEAT LENGTH(list1) TIMES
    {
        IF (list1[i] ≠ list2[i])
        {
            areEqual <- false
        }
        i <- i + 1
    }
}
DISPLAY(areEqual)
```

If `list1 <- [10, 20, 30]` and `list2 <- [10, 25, 30]`, what value is displayed?

(A) error

(B) false

(C) true

(D) null

Question 69

★★★★ HIGH PRIORITY

Cumulative Running Total Generation

Consider the following code segment:

```
original <- [1, 2, 3, 4]
runningTotals <- []
currentSum <- 0

FOR EACH val IN original
{
    currentSum <- currentSum + val
    APPEND(runningTotals, currentSum)
}
```

What are the elements in `runningTotals` after the code segment executes?

(A) [1, 3, 6, 10]

(B) [10, 10, 10, 10]

(C) [0, 1, 3, 6]

(D) [1, 2, 3, 4]

Question 70

★★★★★ VERY IMPORTANT

Concurrent Modification / Deletion Bug

A programmer wants to remove all negative numbers from a list `nums <- [-5, -8, 12, 15]`. The programmer writes:

```
i <- 1
REPEAT LENGTH(nums) TIMES
{
    IF (nums[i] < 0)
    {
        REMOVE(nums, i)
    }
    i <- i + 1
}
```

Why does this code segment FAIL to remove all negative numbers?

(A) The condition nums[i] < 0 cannot evaluate negative numbers.

(B) The REMOVE procedure only works on positive numbers.

(C) REPEAT LENGTH(nums) TIMES causes a syntax error if the list shrinks.

(D) When an element at index i is removed, the remaining elements shift left to fill the gap. Incrementing i immediately afterwards causes the algorithm to skip over the newly shifted element without checking it.

Question 71

★★★★★ VERY IMPORTANT

Procedure Execution with Parameters and RETURN

Consider the following procedure definition and procedure call:

```
PROCEDURE calcFinalPrice(basePrice, discountPercent, taxRate)
{
    discounted <- basePrice - (basePrice * discountPercent / 100)
    finalTotal <- discounted + (discounted * taxRate / 100)
    RETURN finalTotal
}

result <- calcFinalPrice(100, 20, 10)
DISPLAY(result)
```

What value is displayed?

(A) 90

(B) 80

(C) 100

(D) 88

Question 72

★★★★ HIGH PRIORITY

Conditional Return in Procedures

Consider the following procedure intended to return the maximum of two numbers:

```
PROCEDURE findMax(num1, num2)
{
    IF (num1 >= num2)
    {
        RETURN num1
    }
    ELSE
    {
        RETURN num2
    }
}

val <- findMax(35, 42)
DISPLAY(val)
```

What value is displayed?

(A) 35

(B) 42

(C) 77

(D) true

Question 73

★★★★★ VERY IMPORTANT

Benefits of Procedural Abstraction

A team of programmers is developing a large multiplayer video game. They encapsulate the player damage calculation algorithm into a reusable procedure `applyDamage(playerID, hitPoints)`.

Which of the following is the PRIMARY software engineering benefit of using procedural abstraction in this way?

(A) It eliminates the need to test the game for bugs.

(B) It allows the damage calculation logic to be written once and reused across different game events, and allows developers to modify the damage formula in a single place without changing the rest of the game.

(C) It guarantees that the program will run in O(1) constant time regardless of player count.

(D) It converts the game from an interpreted language into binary machine code.

Question 74

★★★★ HIGH PRIORITY

Chained & Nested Procedure Calls

Consider the following procedure definitions:

```
PROCEDURE doubleVal(n)
{
    RETURN n * 2
}

PROCEDURE addFive(n)
{
    RETURN n + 5
}

output <- doubleVal(addFive(3))
DISPLAY(output)
```

What value is displayed?

(A) 16

(B) 8

(C) 11

(D) 13

Question 75

★★★★★ VERY IMPORTANT

Parameters vs. Arguments Distinction

Consider the following code snippet:

```
PROCEDURE computeArea(width, height)
{
    RETURN width * height
}

rectArea <- computeArea(7, 12)
```

Which of the following statements accurately identifies the parameters and arguments in this code?

(A) `computeArea` is a parameter; `rectArea` is an argument.

(B) `width` and 7 are parameters; `height` and 12 are arguments.

(C) `width` and `height` are parameters; 7 and 12 are arguments.

(D) 7 and 12 are parameters; `width` and `height` are arguments.

Question 76

★★★★ HIGH PRIORITY

Side Effects on Global Data Structures

Consider the following code segment:

```
names <- ["Ana", "Ben"]

PROCEDURE addStudent(newName)
{
    APPEND(names, newName)
}

addStudent("Carlos")
addStudent("Diana")
DISPLAY(LENGTH(names))
DISPLAY(names[3])
```

What is displayed after the code segment executes?

(A) 2 Carlos

(B) 4 Diana

(C) 2 Ben

(D) 4 Carlos

Question 77

★★★★★ VERY IMPORTANT

Local Variable Scope vs. Global Scope

Consider the following code segment:

```
x <- 100

PROCEDURE modifyVal(x)
{
    x <- x + 50
    DISPLAY(x)
}

modifyVal(20)
DISPLAY(x)
```

What is displayed after the code segment executes?

(A) 70 70

(B) 20 100

(C) 150 150

(D) 70 100

Question 78

★★★★★ VERY IMPORTANT

Early Return in List Search Procedures

Consider the following procedure intended to search for a target value in a list:

```
PROCEDURE containsValue(aList, target)
{
    FOR EACH item IN aList
    {
        IF (item = target)
        {
            RETURN true
        }
    }
    RETURN false
}

myList <- [12, 45, 78, 92]
result <- containsValue(myList, 45)
DISPLAY(result)
```

What value is displayed, and when does the procedure stop executing?

(A) true; it stops executing immediately when 45 is encountered on the second iteration.

(B) false; it overwrites the result on the final element.

(C) true; it checks all four elements before returning true.

Question 79

★★★★ HIGH PRIORITY

Generalizing Algorithms with Parameters

A school club website has three separate procedures written by a student:
• `calcFreshmanDues()` which multiplies 50 by 0.9
• `calcSophomoreDues()` which multiplies 50 by 0.85
• `calcSeniorDues()` which multiplies 50 by 0.7

Which of the following single procedures best generalizes these three procedures into a flexible, reusable abstraction?

(A) PROCEDURE calcDues(baseAmount, discountRate)
{
RETURN baseAmount * (1 - discountRate)
}

(B) PROCEDURE calcDues(amount)
{
DISPLAY(amount)
}

(C) PROCEDURE calcDues()
{
RETURN 50 * 0.9
}

(D) PROCEDURE calcDues(gradeLevel)
{
RETURN 50
}

Question 80

★★★★★ VERY IMPORTANT

Chained Function Evaluation & Recursion Trace

Consider the following procedure definition:

```
PROCEDURE sumDown(n)
{
    IF (n <= 1)
    {
        RETURN 1
    }
    ELSE
    {
        RETURN n + sumDown(n - 1)
    }
}

ans <- sumDown(4)
DISPLAY(ans)
```

What value is displayed?

(A) 1

(B) 24

(C) 10

(D) 4

Question 81

★★★★★ VERY IMPORTANT

Binary Search Preconditions & Limitations

A programmer needs to search for an employee ID in a list containing 10,000 entries. The programmer considers using a binary search algorithm instead of a linear search algorithm.

Which of the following is a REQUIRED precondition for binary search to function correctly?

(A) The values in the list must all be integers.

(B) The elements in the list must already be sorted in ascending or descending order.

(C) The list must have an even number of elements.

(D) The list must contain only unique values with no duplicates.

Question 82

★★★★★ VERY IMPORTANT

Binary Search Worst-Case Comparisons

A sorted list contains 1,024 names in alphabetical order. In the worst-case scenario, what is the MAXIMUM number of comparisons a binary search algorithm will need to make to determine whether a target name is present in the list?

(A) 100

(B) 11

(C) 512

(D) 1,024

Question 83

★★★★ HIGH PRIORITY

Step-by-Step Binary Search Trace

Consider the following sorted list of 7 integers:

```
`data <- [12, 19, 24, 33, 47, 55, 68]`
```

A binary search algorithm is executed to search for the target value `47`. If the algorithm calculates the middle index as `(low + high) / 2` (rounded down using integer division), which sequence of values from the list is compared with the target before finding it?

(A) 24, then 47

(B) 33, then 47

(C) 12, 19, 24, 33, 47

(D) 33, then 55, then 47

Question 84

★★★★★ VERY IMPORTANT

Random Number Range & Simulation Bounds

In AP CSP pseudocode, `RANDOM(a, b)` returns a randomly selected integer from `a` to `b`, inclusive.

A student writes the following code to simulate rolling two standard 6-sided dice:

```
die1 <- RANDOM(1, 6)
die2 <- RANDOM(1, 6)
diceSum <- die1 + die2
```

What are the minimum and maximum possible values that can be stored in `diceSum`?

(A) Minimum is 2, maximum is 12

(B) Minimum is 2, maximum is 6

(C) Minimum is 1, maximum is 12

(D) Minimum is 0, maximum is 12

Question 85

★★★★ HIGH PRIORITY

Simulating Probabilistic Real-World Events

A developer wants to simulate tossing a fair two-sided coin 1,000 times and record the total number of heads. Let 1 represent heads and 2 represent tails.

Which of the following code segments correctly performs this simulation?

(A)
```
headsCount <- 0
REPEAT 1000 TIMES
{
toss <- RANDOM(0, 2)
IF (toss = 1)
{
headsCount <- headsCount + 1
}
}
```

(B)
```
headsCount <- RANDOM(1, 1000)
```

(C)
```
headsCount <- 0
REPEAT 1000 TIMES
{
toss <- RANDOM(1, 2)
IF (toss = 1)
{
headsCount <- headsCount + 1
}
}
```

(D)
```
headsCount <- 0
REPEAT UNTIL (headsCount = 1000)
{
headsCount <- headsCount + RANDOM(1, 2)
}
```

Question 86

★★★★★ VERY IMPORTANT

Reasonable vs. Unreasonable Algorithmic Time

In computer science, algorithms are categorized by how their execution time scales with the size of the input n.

Which of the following time complexities is considered **UNREASONABLE TIME** for large input sizes?

(A) Linear time: n

(B) Exponential time: 2^n

(C) Logarithmic time: $\log_2(n)$

(D) Quadratic time: n^2

Question 87

★★★★★ VERY IMPORTANT

Heuristic Solutions for Optimization Problems

The Traveling Salesperson Problem (finding the absolute shortest route that visits n cities and returns home) requires checking $(n - 1)! / 2$ possible routes. For $n = 50$ cities, calculating the exact shortest route by brute force would take billions of years on modern supercomputers.

To solve this practical delivery problem, navigation and logistics companies use a **heuristic** algorithm. Which of the following best describes what a heuristic algorithm does?

(A) It runs only when connected to a quantum computer.

(B) It provides a fast, approximate solution that is 'good enough' within reasonable time, even though it is not guaranteed to find the absolute mathematically optimal route.

(C) It converts an NP-hard problem into an undecidable problem.

(D) It guarantees finding the absolute optimal route in O(1) constant time.

Question 88

★★★★ HIGH PRIORITY

Optimization Problems vs. Decision Problems

In computational theory, problems are often classified as either **decision problems** or **optimization problems**.

Which of the following is an example of an **OPTIMIZATION PROBLEM**?

(A) Finding the lowest cost flight itinerary between New York and London among all available airlines.

(B) Determining whether an integer n is a prime number.

(C) Checking whether a user's password contains at least 8 characters.

(D) Determining whether there exists a direct flight between New York and London that costs less than $500.

Question 89

★★★★★ VERY IMPORTANT

Undecidable Problems & The Halting Problem

In 1936, mathematician Alan Turing proved the existence of **undecidable problems**, most famously demonstrating the Halting Problem.

Which of the following statements correctly describes what an **undecidable problem** is in computer science?

(A) A problem that has not yet been solved by programmers but is expected to be solved with faster computers.

(B) A problem that cannot be solved due to lack of RAM memory.

(C) A problem that takes longer than 24 hours to execute on a computer.

(D) A problem for which it is mathematically impossible to construct an algorithm that will always produce a correct yes-or-no answer for every possible input.

Question 90

★★★★★ VERY IMPORTANT

Computational Simulations & Real-World Modeling

Urban traffic planners develop a computer simulation of vehicle movement through a busy downtown intersection to test changing traffic light timings, rather than testing directly on actual city streets.

Which of the following is the PRIMARY advantage of using a computer simulation for this purpose?

(A) The simulation is guaranteed to match real-world driver behavior with 100% mathematical perfection.

(B) Simulations eliminate the need for traffic signals in the real world.

(C) It allows planners to test extreme scenarios (such as blizzards or severe accidents) safely and rapidly without creating real-world traffic gridlock, risking collisions, or incurring high physical costs.

(D) A computer simulation does not require making any simplifying assumptions about reality.

Question 91 · Written Response Scenario

★★★★★ 9 POINTS

School Cafeteria Account & Low-Balance Alert System

A student developer creates an automated mobile payment app called 'CafeteriaPay' to manage meal balances for high school students.

The application maintains a list of student account balances: `accountBalances <- [45.50, 12.00, 3.25, 85.00, 0.00]`.

The core transaction processing algorithm is implemented in the procedure `processTransaction`:

```
PROCEDURE processTransaction(accountList, studentIndex, chargeAmount)
{
    IF (studentIndex >= 1 AND studentIndex <= LENGTH(accountList))
    {
        currentBal <- accountList[studentIndex]
        IF (currentBal >= chargeAmount)
        {
            accountList[studentIndex] <- currentBal - chargeAmount
            IF (accountList[studentIndex] < 5.00)
            {
                sendLowBalanceAlert(studentIndex)
            }
            RETURN true
        }
        ELSE
        {
            logDeclinedCharge(studentIndex, chargeAmount)
            RETURN false
        }
    }
    RETURN false
}
```

The app calls `processTransaction(accountBalances, 3, 2.50)` when student #3 purchases a lunch item.

(a) Identify the data abstraction used in this program, describe the type of data it stores, and explain how using this list manages complexity compared to creating separate variables for each student. **(2 points)**

(b) Identify the procedure name, list its parameters, and explain in detail the algorithm implemented inside the procedure, including how sequencing, selection, and data updates are coordinated. **(3 points)**

(c) Describe two distinct calls to `processTransaction` that execute different conditional branches, specifying the arguments passed, the condition evaluated, and the resulting output/state for each call. **(2 points)**

(d) Explain how an index out-of-bounds error could occur in this system and explain the specific defensive programming check the developer included to prevent it. **(2 points)**

Question 92 · Written Response Scenario

★★★★★ 9 POINTS

Fitness Tracker Step Goal Analyzer

A student creates a health companion app called 'StrideSync' that records daily steps over a 7-day week and evaluates whether the user achieved their weekly fitness targets.

The weekly data is stored in a list: `weeklySteps <- [8400, 10200, 9500, 6100, 11400, 12000, 7800]`.

The progress evaluation algorithm is encapsulated in procedure `evaluateWeeklyProgress`:

```
PROCEDURE evaluateWeeklyProgress(stepsList, dailyGoal)
{
    totalSteps <- 0
    daysGoalMet <- 0

    FOR EACH dayCount IN stepsList
    {
        totalSteps <- totalSteps + dayCount
        IF (dayCount >= dailyGoal)
        {
            daysGoalMet <- daysGoalMet + 1
        }
    }

    avgSteps <- totalSteps / LENGTH(stepsList)

    IF (daysGoalMet >= 5 AND avgSteps >= dailyGoal)
    {
        RETURN "Goal Achieved! Outstanding consistency."
    }
    ELSE
    {
        RETURN "Goal Incomplete. Keep pushing next week!"
    }
}
```

(a) Identify the list used in the program, explain its role as a data abstraction, and identify what would occur if the user tracked 30 days instead of 7. **(2 points)**

(b) Explain the algorithm inside `evaluateWeeklyProgress`, detailing how it utilizes sequencing, selection, and iteration to compute both total steps and goal consistency. **(3 points)**

(c) Calculate the exact values of `totalSteps`, `daysGoalMet`, and `avgSteps` when `evaluateWeeklyProgress(weeklySteps, 10000)` is executed, and state the final return value. **(2 points)**

(d) Provide two distinct procedure calls with different arguments that result in different string return messages, specifying the inputs and resulting outputs. **(2 points)**

Question 93 · Written Response Scenario

★★★★★ 9 POINTS

Warehouse Inventory Restock Prioritization Algorithm

An e-commerce fulfillment warehouse utilizes an automated inventory robot management system. The system maintains parallel lists tracking items, current stock, and minimum thresholds:

`stockLevels <- [14, 5, 42, 2, 80]`
`reorderLimits <- [20, 15, 30, 10, 50]`

The restocking algorithm scans items and populates a restock queue:

```
PROCEDURE generateRestockOrders(stock, limits, urgencyFactor)
{
    urgentOrders <- []
    i <- 1

    REPEAT LENGTH(stock) TIMES
    {
        currentStock <- stock[i]
        threshold <- limits[i]

        IF (currentStock < threshold)
        {
            deficit <- threshold - currentStock
            IF (deficit >= urgencyFactor)
            {
                APPEND(urgentOrders, i)
            }
        }
        i <- i + 1
    }
    RETURN urgentOrders
}
```

(a) Explain how parallel lists function as a data abstraction in this system, and state the relationship between indices in `stockLevels` and `reorderLimits`. **(2 points)**

(b) Explain the algorithmic logic inside `generateRestockOrders`, detailing how it identifies which items require urgent restocking. **(3 points)**

(c) Determine the exact list returned when `generateRestockOrders(stockLevels, reorderLimits, 7)` is executed, showing work for each index. **(2 points)**

(d) Describe what logical error would occur if the instruction `i <- i + 1` was placed inside the inner `IF (deficit >= urgencyFactor)` block instead of at the end of the loop. **(2 points)**

Question 94 · Written Response Scenario

★★★★★ 9 POINTS

Autonomous Delivery Drone Route Safety Verification

A commercial delivery company programs autonomous parcel drones using a flight controller. The flight path consists of a list of planned waypoint altitudes in meters:

`flightAltitudes <- [120, 145, 95, 110, 80, 130]`

The drone safety protocol executes before takeoff:

```
PROCEDURE verifyFlightSafety(altitudeList, minAltitude, maxAltitude)
{
    isSafe <- true
    violatingWaypoints <- 0

    FOR EACH alt IN altitudeList
    {
        IF (alt < minAltitude OR alt > maxAltitude)
        {
            isSafe <- false
            violatingWaypoints <- violatingWaypoints + 1
        }
    }

    IF (isSafe = true)
    {
        RETURN "Flight Path Approved"
    }
    ELSE
    {
        RETURN concat("Safety Violation: Ineligible Waypoints = ", violatingWaypoints)
    }
}
```

(a) Identify the data abstraction in this system and explain why storing altitudes in a list is necessary for multi-waypoint navigation. **(2 points)**

(b) Explain the algorithm in `verifyFlightSafety`, identifying how Boolean flag tracking and conditional logic verify the safety envelope. **(3 points)**

(c) Determine the exact return string when `verifyFlightSafety(flightAltitudes, 90, 150)` is executed. Show each evaluated waypoint. **(2 points)**

(d) Provide two distinct calls to `verifyFlightSafety` with different parameters that execute different branches of the final conditional. **(2 points)**

Question 95 · Written Response Scenario

★★★★★ 9 POINTS

Mobile Quiz Battle Streak & Scoring Engine

A mobile education app incorporates a head-to-head 1v1 quiz battle feature. After a match, the user's answers are recorded as a list of Booleans where `true` indicates a correct answer and `false` indicates an incorrect answer:

`matchAnswers <- [true, true, true, false, true, true]`

The score calculation algorithm awards base points and streak multipliers:

```
PROCEDURE computeQuizScore(answerList, pointsPerQuestion, streakBonus)
{
    totalScore <- 0
    currentStreak <- 0

    FOR EACH isCorrect IN answerList
    {
        IF (isCorrect = true)
        {
            currentStreak <- currentStreak + 1
            totalScore <- totalScore + pointsPerQuestion
            IF (currentStreak >= 3)
            {
                totalScore <- totalScore + streakBonus
            }
        }
        ELSE
        {
            currentStreak <- 0
        }
    }
    RETURN totalScore
}
```

(a) Identify the list data abstraction and explain why a list of Booleans is suitable for recording match response accuracy. **(2 points)**

(b) Explain the algorithm in `computeQuizScore`, focusing on how streak counters are incremented, reset, and used to calculate bonuses. **(3 points)**

(c) Calculate the exact value returned by `computeQuizScore(matchAnswers, 100, 50)`, showing the values of `currentStreak` and `totalScore` after each question. **(2 points)**

(d) Describe how the score would change if question #4 was `true` instead of `false`, and explain why this single change has a compounding effect on total score. **(2 points)**

Question 96 · Written Response Scenario

★★★★★ 9 POINTS

Hospital Emergency Room Patient Triage Priority Queue

A hospital emergency department utilizes an electronic triage board. When patients arrive, triage nurses assign an emergency acuity score from 1 (resuscitation/critical) to 5 (non-urgent):

`patientAcuity <- [3, 4, 1, 2, 5, 1, 3]`

The medical dispatch algorithm scans the queue to locate the most critical patients:

```
PROCEDURE findMostCriticalPatients(acuityList, targetAcuity)
{
    matchingPatientIndices <- []
    i <- 1

    REPEAT LENGTH(acuityList) TIMES
    {
        IF (acuityList[i] = targetAcuity)
        {
            APPEND(matchingPatientIndices, i)
        }
        i <- i + 1
    }
    RETURN matchingPatientIndices
}
```

(a) Identify the list data abstraction and explain how it represents patient priority in the triage queue. **(2 points)**

(b) Explain the algorithmic logic inside `findMostCriticalPatients`, detailing how it performs a linear search across the list. **(3 points)**

(c) Determine the exact list returned when `findMostCriticalPatients(patientAcuity, 1)` is called, identifying which patients require immediate attention. **(2 points)**

(d) Explain what would occur if `findMostCriticalPatients` was called with `targetAcuity = 6` (a severity level not present in the list). **(2 points)**

Question 97 · Written Response Scenario

★★★★★ 9 POINTS

Smart Home Energy Conservation & Peak-Hours Load Shedding

An IoT smart-home energy management controller tracks electrical power draw. The controller records hourly power consumption in kilowatt-hours (kWh) across a 24-hour day in a list:

`hourlyUsage <- [1.2, 0.8, 0.7, 0.6, 0.9, 1.5, 3.2, 4.1, 2.8, 2.5, 2.1, 2.4, 2.6, 2.9, 3.1, 3.8, 5.2, 6.1, 5.8, 4.2, 3.5, 2.8, 1.9, 1.4]`

To prevent utility peak-demand surcharges, the controller executes a load-shedding procedure:

```
PROCEDURE evaluatePeakConsumption(usageList, peakStartHour, peakEndHour, limitKWh)
{
    peakTotal <- 0
    hoursOverLimit <- 0
    i <- peakStartHour

    REPEAT UNTIL (i > peakEndHour)
    {
        hourlyDraw <- usageList[i]
        peakTotal <- peakTotal + hourlyDraw
        IF (hourlyDraw > limitKWh)
        {
            hoursOverLimit <- hoursOverLimit + 1
        }
        i <- i + 1
    }

    IF (hoursOverLimit >= 2)
    {
        triggerLoadShedding()
        RETURN "High Demand: Load Shedding Activated"
    }
    ELSE
    {
        RETURN "Normal Demand: Grid Balanced"
    }
}
```

(a) Identify the list data abstraction and explain how 1-based indexing maps to the 24 hours of a day. **(2 points)**

(b) Explain the algorithm in `evaluatePeakConsumption`, detailing how the `REPEAT UNTIL` loop restricts evaluation strictly to peak hours. **(3 points)**

(c) Assuming peak hours run from Hour 16 to Hour 19 (inclusive) with `limitKWh = 5.0`, trace the algorithm and determine the returned status string. **(2 points)**

(d) Explain how changing `limitKWh` to 6.5 would affect the conditional decision and resulting output for the same peak window. **(2 points)**

Question 98 · Written Response Scenario

★★★★★ 9 POINTS

E-Commerce Shopping Cart Discount & Free Shipping Calculator

An online retail platform calculates checkout totals for shopping carts. An active order stores item prices in a list: `cartItems <- [25.00, 15.50, 42.00, 8.50, 60.00]`.

The pricing engine calculates subtotals, promotional discounts, and shipping eligibility:

```
PROCEDURE computeOrderTotal(itemsList, promoCode, isPrimeMember)
{
    rawSubtotal <- 0
    FOR EACH price IN itemsList
    {
        rawSubtotal <- rawSubtotal + price
    }

    discountedSubtotal <- rawSubtotal
    IF (promoCode = "SAVE20")
    {
        discountedSubtotal <- rawSubtotal * 0.80
    }

    shippingFee <- 10.00
    IF (isPrimeMember = true OR discountedSubtotal >= 100.00)
    {
        shippingFee <- 0.00
    }

    finalTotal <- discountedSubtotal + shippingFee
    RETURN finalTotal
}
```

(a) Identify the list data abstraction and explain why a list is required when shopping carts can hold any number of items. **(2 points)**

(b) Explain the algorithm in `computeOrderTotal`, showing how sequencing determines subtotal, discount, shipping, and final total. **(3 points)**

(c) Calculate the exact value returned by `computeOrderTotal(cartItems, "SAVE20", false)`. Show subtotal, discount, and shipping calculations. **(2 points)**

(d) Provide two distinct procedure calls that evaluate different branches of the shipping conditional, specifying the parameters and resulting shipping fees. **(2 points)**

Question 99 · Written Response Scenario

★★★★★ 9 POINTS

Cybersecurity Network Traffic Anomaly & DDoS Detection

A network security intrusion detection system (IDS) monitors packet throughput across an enterprise gateway router. It records the number of packets received per second over a 10-second monitoring window in a list:

`trafficRates <- [450, 480, 510, 490, 520, 2400, 3100, 2800, 500, 470]`

The traffic inspection algorithm flags sudden distributed denial-of-service (DDoS) spikes:

```
PROCEDURE detectTrafficAnomaly(ratesList, baselineMax, spikeThreshold)
{
    spikeCount <- 0
    maxObserved <- ratesList[1]

    FOR EACH rate IN ratesList
    {
        IF (rate > maxObserved)
        {
            maxObserved <- rate
        }
        IF (rate >= baselineMax * spikeThreshold)
        {
            spikeCount <- spikeCount + 1
        }
    }

    IF (spikeCount >= 3)
    {
        alertNetworkAdmin(maxObserved)
        RETURN concat("CRITICAL ALERT: DDoS Attack Detected. Peak = ", maxObserved)
    }
    ELSE
    {
        RETURN "Traffic Normal: No Anomaly Detected"
    }
}
```

(a) Identify the list data abstraction and explain how it enables time-series anomaly detection. **(2 points)**

(b) Explain the algorithm in `detectTrafficAnomaly`, showing how it tracks peak traffic and detects anomalous bursts. **(3 points)**

(c) Trace the execution when `detectTrafficAnomaly(trafficRates, 600, 4)` is called. State the values of `spikeCount` and `maxObserved`, and the return message. **(2 points)**

(d) Explain why single-packet spikes (e.g. `spikeCount = 1`) are filtered out and do not trigger a critical alert, referencing the role of false positive mitigation. **(2 points)**

Question 100 · Written Response Scenario

★★★★★ 9 POINTS

City Bike-Share Station Balancing & Redistribution Algorithm

A metropolitan transit department operates an automated municipal bike-share network. The system monitors the count of available bicycles across 5 downtown docking stations:

`stationBikes <- [18, 3, 22, 1, 14]`

Each station has an optimal target capacity of 10 bicycles. To direct logistics vans, the central dispatcher runs an automated rebalancing procedure:

```
PROCEDURE planVanRedistribution(bikeList, targetCapacity)
{
    emptyStations <- []
    totalDeficit <- 0
    i <- 1

    REPEAT LENGTH(bikeList) TIMES
    {
        currentCount <- bikeList[i]
        IF (currentCount < targetCapacity)
        {
            shortfall <- targetCapacity - currentCount
            totalDeficit <- totalDeficit + shortfall
            IF (currentCount <= 3)
            {
                APPEND(emptyStations, i)
            }
        }
        i <- i + 1
    }

    IF (LENGTH(emptyStations) >= 2)
    {
        dispatchVan(emptyStations, totalDeficit)
        RETURN concat("Van Dispatched: Critical Stations = ", LENGTH(emptyStations))
    }
    ELSE
    {
        RETURN "Stations Balanced: No Van Required"
    }
}
```

(a) Identify the list data abstraction and explain how station numbers correspond to list indices in the bike-share system. **(2 points)**

(b) Explain the algorithm in `planVanRedistribution`, detailing how it calculates total system deficit and identifies critically empty stations. **(3 points)**

(c) Determine the exact contents of `emptyStations`, the value of `totalDeficit`, and the returned string when `planVanRedistribution(stationBikes, 10)` executes. **(2 points)**

(d) Describe what change would occur if station #4 received 5 bicycles prior to running the algorithm, explaining the effect on van dispatching. **(2 points)**

SECTION II ❖ EXAMINATION KEY & SOLUTIONS

Answers, Worked Solutions & Scoring Rubrics

The following pages contain the complete 90-question Multiple Choice Answer Key Matrix, comprehensive step-by-step worked solutions with distractor failure analysis, and official 9-point College Board scoring rubrics for all 10 Create Performance Task Written Response questions.

Section I Answer Key Matrix

Quick-reference answer matrix for Multiple Choice Questions 1 through 90. All questions are weighted equally.

Q1 – Q23		Q24 – Q46		Q47 – Q68		Q69 – Q90	
Q01	A	Q24	A	Q47	A	Q69	A
Q02	C	Q25	B	Q48	B	Q70	D
Q03	D	Q26	C	Q49	B	Q71	D
Q04	D	Q27	C	Q50	C	Q72	B
Q05	A	Q28	D	Q51	C	Q73	B
Q06	C	Q29	A	Q52	D	Q74	A
Q07	C	Q30	B	Q53	D	Q75	C
Q08	B	Q31	C	Q54	B	Q76	D
Q09	D	Q32	C	Q55	C	Q77	D
Q10	D	Q33	A	Q56	C	Q78	A
Q11	C	Q34	D	Q57	A	Q79	A
Q12	C	Q35	B	Q58	D	Q80	C
Q13	A	Q36	C	Q59	D	Q81	B
Q14	A	Q37	A	Q60	C	Q82	B
Q15	D	Q38	A	Q61	C	Q83	D
Q16	D	Q39	B	Q62	B	Q84	A
Q17	C	Q40	D	Q63	B	Q85	C
Q18	A	Q41	C	Q64	D	Q86	B
Q19	A	Q42	A	Q65	D	Q87	B
Q20	B	Q43	A	Q66	C	Q88	A
Q21	C	Q44	B	Q67	B	Q89	D
Q22	D	Q45	B	Q68	B	Q90	C
Q23	A	Q46	D				

Question 01 · Variable Assignment & Sequential Execution

Priority ★★★★★

CORRECT ANSWER: (A) 30 30

- Follow the sequential assignment statement by statement:
- • Line 1: `first` is assigned the value 15.
- • Line 2: `second` is assigned the value 30.
- • Line 3: `first <- second` copies the current value of `second` (30) into `first`. Now `first` is 30, and the original value (15) is overwritten and lost.
- • Line 4: `second <- first` copies the current value of `first` (which is now 30) into `second`. `second` remains 30.
- • Lines 5–6: Both variables evaluate to 30. Therefore, the display output is `30 30`.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(B):** Assumes `second` overwrites backwards or reverts both variables to the initial value of `first` (15).
✕ **(C):** Assumes `first <- second` does not execute or fails to overwrite `first`.
✕ **(D):** Assumes the code successfully swaps the two values. A third temporary variable (`temp`) is required to perform a swap without data loss.

Question 02 · Variable Swapping Algorithm

Priority ★★★★★

CORRECT ANSWER: (C) temp <- x x <- y y <- temp

- Swapping requires a 3-step sequence using a temporary holding variable to prevent overwriting:
- • Step 1: Save x's initial value into `temp` (`temp <- x`). Now `temp` safely preserves x.
- • Step 2: Overwrite x with y's value (`x <- y`). Now x holds y's original value.
- • Step 3: Overwrite y with the preserved value from `temp` (`y <- temp`).
- Result: x has y's original value, and y has x's original value.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** `temp` has not been initialized with any value, leading to an undefined or null assignment into `x`.
✕ **(B):** Line 2 overwrites `y` with `x`, and then Line 3 sets `x` to `y` (which is already `x`). Both variables end up storing `x`'s original value.
✕ **(D):** Line 1 immediately overwrites `x` with `y`, so `x`'s original value is permanently lost before `temp` can save it. Both variables end up with `y`'s original value.

Question 03 · Modulo Operator & Clock Arithmetic

Priority ★★★★

CORRECT ANSWER: (D) 12

- Trace the arithmetic calculation and conditional logic:
- • `currentHour + elapsedHours` evaluates to `9 + 15 = 24`.
- • Compute `24 MOD 12`: 24 divided by 12 equals 2 with a remainder of 0. Thus, `newHour` is assigned 0.
- • The condition `IF (newHour = 0)` evaluates to `true`.
- • The block inside the `IF` executes: `newHour <- 12`.
- • Therefore, the output displayed is `12` (representing 12:00 on a standard clock, rather than 0:00).

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Fails to execute the `IF (newHour = 0)` conditional check which replaces 0 with 12 on a standard 12-hour clock.
✕ **(B):** Incorrectly computes `24 MOD 12` as 6 (confusing 15 - 9 with the modulo remainder).
✕ **(C):** Displays the un-moduloed total sum (24) rather than applying the remainder operator.

Question 04 · Arithmetic Operator Precedence & Evaluation

Priority ★★★★

CORRECT ANSWER: (D) 14

- Apply the standard order of operations (Multiplication, Division, and Modulo precede Addition and Subtraction; operations of equal precedence evaluate left-to-right):
- • Step 1: Evaluate `q * r` -> `6 * 2 = 12`.
- • Step 2: Evaluate `14 MOD 4` -> 14 divided by 4 is 3 with remainder 2.
- • Step 3: Substitute the calculated terms: `p + 12 - 2` -> `4 + 12 - 2`.
- • Step 4: Perform addition and subtraction from left to right: `4 + 12 = 16`, then `16 - 2 = 14`.
- • Therefore, `DISPLAY(result)` outputs `14`.

WHY OTHER CHOICES ARE INCORRECT:

- ✕ **(A):** Forgets to subtract the remainder `14 MOD 4`, outputting `4 + 12 = 16`.
- ✕ **(B):** Evaluates `(p + q) * r` first as `(4 + 6) * 2 = 20`, then subtracts 2 to get 18, violating operator precedence.
- ✕ **(C):** Incorrectly calculates `14 MOD 4` as 4 or miscalculates the final subtraction as `16 - 4 = 12`.

Question 05 · Sequential Accumulator Updates

Priority ★★★★

CORRECT ANSWER: (A) 55

- Trace the variable states sequentially step-by-step:
- • Line 1: `total` is initialized to 20.
- • Line 2: `factor` is initialized to 3.
- • Line 3: `total <- total * factor` -> `20 * 3 = 60`. Now `total` is 60.
- • Line 4: `factor <- factor + 2` -> `3 + 2 = 5`. Now `factor` is 5.
- • Line 5: `total <- total - factor` -> `60 - 5 = 55`. Now `total` is 55.
- • Line 6: `DISPLAY(total)` prints `55`.

WHY OTHER CHOICES ARE INCORRECT:

- ✕ **(B):** Subtracts the original `factor` (3) instead of the updated `factor` (5): `60 - 3 = 57`.
- ✕ **(C):** Forgets to subtract `factor` on Line 5, keeping the value at 60.
- ✕ **(D):** Incorrectly increments `factor` by 4 instead of 2 before subtracting: `60 - 7 = 53`.

Question 06 · Divisibility Testing Using Modulo

Priority ★★★★★

CORRECT ANSWER: (C) (n MOD 4 = 0) AND (n MOD 6 = 0)

- Analyze divisibility testing using the remainder operator (`MOD`):
- • A number n is evenly divisible by k if and only if the remainder when dividing n by k is 0 (`n MOD k = 0`).
- • For n to be divisible by BOTH 4 and 6, both conditions must be true simultaneously, requiring the logical `AND` operator: `(n MOD 4 = 0) AND (n MOD 6 = 0)`.
- • For example, if `n = 12`, `12 MOD 4 = 0` (true) and `12 MOD 6 = 0` (true), so the expression evaluates to `true`.

WHY OTHER CHOICES ARE INCORRECT:

- ✕ **(A):** Using `OR` evaluates to `true` if `n` is divisible by 4 OR by 6 (e.g., `n = 8` is divisible by 4 but not 6, yet `OR` would incorrectly evaluate to `true`).
- ✕ **(B):** Requires `n` to be divisible by 24. While numbers divisible by 24 are divisible by both 4 and 6, the lowest common multiple of 4 and 6 is 12 (e.g., 12 and 36 are divisible by both 4 and 6, but not by 24).
- ✕ **(D):** Checks whether `n` is divisible by 10 (sum of 4 and 6), which is mathematically unrelated.

Question 07 · String Concatenation & Substring Operations

Priority ★★★★

CORRECT ANSWER: (C) taba

- Trace the string concatenation and 1-based substring extraction:
- • Line 3: `fullWord <- concat("data", "base")` results in `"database"`.
- • Note the 1-based indices for `"database"`:
- Index 1: 'd', Index 2: 'a', Index 3: 't', Index 4: 'a', Index 5: 'b', Index 6: 'a', Index 7: 's', Index 8: 'e'.
- • Line 4: `substring(fullWord, 3, 4)` starts at index 3 ('t') and extracts 4 consecutive characters:
- - Index 3: 't'
- - Index 4: 'a'
- - Index 5: 'b'
- - Index 6: 'a'
- • The extracted substring is `"taba"`.
- • Line 5: `DISPLAY(part)` outputs `taba`.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Assumes 0-based indexing starting at index 2 ('a') rather than 1-based index 3 ('t').
✕ **(B):** Extracts `prefix` ("data") which starts at index 1, not index 3.
✕ **(D):** Extracts `suffix` ("base") which starts at index 5, not index 3.

Question 08 · Equivalent Mathematical Expressions in Pseudocode

Priority ★★★★

CORRECT ANSWER: (B) (q1 + q2 + q3) / 3 + 5

- Examine the mathematical structure of the calculation:
- • The average of three scores is `(q1 + q2 + q3) / 3`.
- • The parentheses around `q1 + q2 + q3` ensure that the three scores are summed before dividing by 3.
- • Adding the 5-point curve to the average gives `((q1 + q2 + q3) / 3) + 5`.
- • Because division has higher precedence than addition, `(q1 + q2 + q3) / 3 + 5` correctly computes the division first and then adds 5.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Divides the sum by `(3 + 5) = 8`, which computes an incorrect scaled fraction rather than a curved 3-score average.
✕ **(C):** Adds 5 inside the numerator before dividing by 3, which curves the final grade by only `5 / 3 ≈ 1.67` points instead of 5 full points.
✕ **(D):** Due to operator precedence, `q3 / 3` is evaluated first, effectively computing `q1 + q2 + (q3/3) + 5` instead of dividing the entire sum.

Question 09 · Instruction Sequencing & Side Effects

Priority ★★★★★

CORRECT ANSWER: (D) Segment 1: x = 30, y = 10; Segment 2: x = 0, y = -10

- Trace both segments sequentially starting from `x = 10, y = 20`:
- • Segment 1:
- - Line 1: `x <- x + y` -> `10 + 20 = 30`. (x is now 30, y is 20).
- - Line 2: `y <- x - y` -> `30 - 20 = 10`. (x remains 30, y is now 10).
- - Final Segment 1: `x = 30, y = 10` (the arithmetic trick to swap without a temporary variable).
- • Segment 2:
- - Line 1: `y <- x - y` -> `10 - 20 = -10`. (x is 10, y is now -10).
- - Line 2: `x <- x + y` -> `10 + (-10) = 0`. (x is now 0, y remains -10).
- - Final Segment 2: `x = 0, y = -10`.
- • Therefore, Option A is correct.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Computes `y <- y - x` instead of `x - y` in Segment 1.
✕ **(B):** Assumes changing the order of instructions does not change the resulting values (violating fundamental algorithmic sequencing).
✕ **(C):** Reverses the assignments of `x` and `y` in Segment 1.

Question 10 · Digit Extraction Using Modulo and Division

Priority ★★★★

CORRECT ANSWER: (D) num / 10

- Analyze integer division vs modulo for digit extraction:
- • For any two-digit number `num = 10 × tens + units` (e.g., 47 = 10 × 4 + 7):
- • Dividing `num` by 10 using integer division: `47 / 10 = 4` (the fractional part 0.7 is discarded). This isolates the tens digit.
- • In contrast, `num MOD 10` gives the remainder: `47 MOD 10 = 7`, which isolates the units (ones) digit.
- • Therefore, `num / 10` correctly extracts the tens digit.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** `num - 10` simply subtracts 10 (e.g., `47 - 10 = 37`), which does not isolate the tens digit.
✕ **(B):** `num MOD 10` is 7; dividing 7 by 10 yields 0 in integer division.
✕ **(C):** `num MOD 10` extracts the units (ones) digit (e.g., `47 MOD 10 = 7`), not the tens digit.

Question 11 · Compound Boolean Expressions with AND, OR, NOT

Priority ★★★★★

CORRECT ANSWER: (C) false

- Trace the Boolean evaluation step-by-step:
- • Step 1: Evaluate `isWeekend OR isHoliday` -> `true OR false` evaluates to `true`.
- • Step 2: Evaluate `NOT hasHomework` -> `NOT true` evaluates to `false`.
- • Step 3: Evaluate the `AND` conjunction: `(true) AND (false)`.
- • In Boolean logic, `true AND false` evaluates to `false`.
- • Therefore, `canPlayVideoGames` is assigned `false`, and `DISPLAY` outputs `false`.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Incorrectly treats `(NOT hasHomework)` as true or evaluates the `OR` expression as sufficient to make the entire statement true.
✕ **(B):** Boolean expressions always evaluate to either `true` or `false`, never `null`.
✕ **(D):** The expression is syntactically valid in AP CSP pseudocode and produces no runtime error.

Question 12 · De Morgan's Laws & Condition Simplification

Priority ★★★★★

CORRECT ANSWER: (C) (score <= 85) AND (attendance >= 90)

- Apply De Morgan's Law for distributing a negation across a logical disjunction:
- • De Morgan's Law states: `NOT (A OR B)` is logically equivalent to `(NOT A) AND (NOT B)`.
- • Step 1: Negate the first sub-expression: `NOT (score > 85)` is `(score <= 85)`.
- • Step 2: Change `OR` to `AND`.
- • Step 3: Negate the second sub-expression: `NOT (attendance < 90)` is `(attendance >= 90)`.
- • Combining gives: `(score <= 85) AND (attendance >= 90)`.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Fails to flip the logical operator from `OR` to `AND` when distributing the `NOT`.
✕ **(B):** Keeps the `OR` operator between the negated terms instead of changing to `AND`.
✕ **(D):** Uses strict inequalities (`<` and `>`) instead of inclusive inequalities (`<=` and `>=`), incorrectly omitting the boundary values 85 and 90.

Question 13 · Truth Table Evaluation

Priority ★★★★

CORRECT ANSWER: (A) p AND (NOT q)

- Analyze the required truth conditions:
- • We need an expression that is `true` when `p = true` AND `q = false`, and `false` for all other combinations.
- • When `p` is `true`, `p` is `true`.
- • When `q` is `false`, `NOT q` evaluates to `true`.
- • The conjunction `p AND (NOT q)` evaluates to `true AND true = true`.
- • For any other combination (e.g., `p = true, q = true`), `NOT q` is `false`, making the expression `false`.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(B):** Evaluates to `true` when `p` is `false` and `q` is `true`.
✕ **(C):** Evaluates to `true` whenever `p` and `q` are not both `true` (e.g., evaluates to `true` when both `p` and `q` are `false`).
✕ **(D):** Evaluates to `true` whenever `p` is `true`, regardless of `q` (e.g., when `p = true, q = true`, `p OR (NOT q)` is `true`).

Question 14 · Operator Precedence in Boolean Expressions

Priority ★★★★

CORRECT ANSWER: (A) true

- Evaluate according to logical operator precedence (`NOT` > `AND` > `OR`):
- • Step 1: `b AND c` has higher precedence, so it evaluates first: `true AND true` evaluates to `true`.
- • Step 2: Substitute into the `OR` expression: `a OR true -> false OR true`.
- • Step 3: `false OR true` evaluates to `true`.
- • Therefore, `DISPLAY(result)` prints `true`.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(B):** The expression is fully defined and produces no undefined error.
✕ **(C):** Boolean variables produce `true` or `false`, never `null`.
✕ **(D):** Incorrectly evaluates left-to-right without precedence: `(a OR b)` is `true`, but if someone incorrectly evaluated `(false OR true) AND false` they might get false.

Question 15 · Boundary Value Analysis in Relational Expressions

Priority ★★★★★

CORRECT ANSWER: (D) Condition 1 and Condition 2 evaluate to the exact same Boolean result for all possible integer values of studentAge.

- Analyze integer boundaries for both conditions:
- • In Condition 1: `studentAge >= 14` means `studentAge` can be 14, 15, 16, 17, 18, ... and `studentAge <= 18` means up to 18.
- The allowed integer values are {14, 15, 16, 17, 18}.
- • In Condition 2: For integers, the strictly smallest integer greater than 13 (`studentAge > 13`) is 14. The strictly largest integer less than 19 (`studentAge < 19`) is 18.
- The allowed integer values are also {14, 15, 16, 17, 18}.
- • Because `studentAge` is constrained to integers, both conditions produce identical Boolean outputs for every integer input.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Both expressions operate on numerical values; the prompt explicitly specifies `studentAge` is an integer.
✕ **(B):** Strict inequalities `> 13` and `< 19` strictly exclude 13 and 19.
✕ **(C):** For integers, `studentAge > 13` includes 14, and `studentAge < 19` includes 18. They are not excluded.

Question 16 · Double Negation & Boolean Simplification

Priority ★★★★

CORRECT ANSWER: (D) p OR q

- Apply De Morgan's Law and the rule of Double Negation:
- • De Morgan's Law states: `NOT (A AND B)` is equivalent to `(NOT A) OR (NOT B)`.
- • Let `A = NOT p` and `B = NOT q`.
- • Then `NOT (NOT p AND NOT q)` becomes `(NOT (NOT p)) OR (NOT (NOT q))`.
- • Double negation states that `NOT (NOT x)` is simply `x`.
- • Therefore, `(NOT (NOT p)) OR (NOT (NOT q))` simplifies to `p OR q`.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Incorrectly simplifies to `p AND q` without applying De Morgan's operator flip from `AND` to `OR`.
✕ **(B):** `NOT (p OR q)` is equivalent to `NOT p AND NOT q`, which is the exact negation of the original expression.
✕ **(C):** `NOT p OR NOT q` is equivalent to `NOT (p AND q)`, not `NOT (NOT p AND NOT q)`.

Question 17 · Range Exclusion vs Inclusion Logic

Priority ★★★★★

CORRECT ANSWER: (C) (temp < 15) OR (temp > 85)

- Analyze the problem specification:
- • The condition specifies 'strictly below 15°C' -> `temp < 15`.
- • 'strictly above 85°C' -> `temp > 85`.
- • The word 'either... or' indicates that meeting either condition flags the item, requiring the logical `OR` operator.
- • Therefore, the correct condition is `(temp < 15) OR (temp > 85)`.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Using `AND` requires a number to be both less than 15 AND greater than 85 at the same time, which is mathematically impossible (evaluates to `false` for all temperatures).
✕ **(B):** Uses inclusive inequalities (`<=` and `>=`), which would flag valid items at exactly 15°C and 85°C.
✕ **(D):** Negates the expression, which evaluates to `true` when the temperature is normal (safe range between 15°C and 85°C).

Question 18 · Boolean Flags in Search Logic

Priority ★★★★

CORRECT ANSWER: (A) true

- Trace the Boolean flag updates:
- • Line 1: `found` is initialized to `false`.
- • Check 1: `val1 = 42 -> 18 = 42` is `false`. The first `IF` body does not execute.
- • Check 2: `val2 = 42 -> 42 = 42` is `true`. The second `IF` body executes: `found <- true`.
- • Check 3: `val3 = 42 -> 99 = 42` is `false`. The third `IF` body does not execute, so `found` remains `true`.
- • `DISPLAY(found)` outputs `true`.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(B):** Assumes `found` reverts to `false` because `val3` is not equal to 42. Since there is no `ELSE { found <- false }`, the flag remains `true` once set.
✕ **(C):** The variable `found` is a Boolean (`true`/`false`), not the integer 42.
✕ **(D):** The syntax and logic are completely valid.

Question 19 · Complex Expression Evaluation with Multiple Variables

Priority ★★★★

CORRECT ANSWER: (A) false

- Evaluate the expression step-by-step:
- • Left side of OR: `(x * 2 > y)` -> `(5 * 2 > 12)` -> `(10 > 12)` -> `false`.
- • Right side of OR: `(z / 2 = 10 AND y < x)`:
- - First part: `(20 / 2 = 10)` -> `(10 = 10)` -> `true`.
- - Second part: `(y < x)` -> `(12 < 5)` -> `false`.
- - Conjunction: `true AND false` -> `false`.
- • Combine with OR: `(false) OR (false)` -> `false`.
- • Therefore, `DISPLAY(condition)` prints `false`.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(B):** Outputs a numerical result rather than the Boolean outcome of the relational expression.
✕ **(C):** Incorrectly evaluates `(z / 2 = 10)` as sufficient without recognizing that `y < x` is `false`, making the entire `AND` clause `false`.
✕ **(D):** Boolean expressions evaluate to `true` or `false`.

Question 20 · Logical Contradiction Bug in Conditionals

Priority ★★★★★

CORRECT ANSWER: (B) No single number can be simultaneously less than 2 and greater than 8, making the AND condition a logical contradiction that always evaluates to false.

- Analyze the logical contradiction:
- • The logical operator `AND` requires both conjuncts to be `true` at the exact same moment for the overall condition to be `true`.
- • For any real number `temp`, if `temp < 2`, then `temp` cannot be greater than 8.
- • Conversely, if `temp > 8`, `temp` cannot be less than 2.
- • Thus, `(temp < 2 AND temp > 8)` is a logical impossibility (contradiction) that evaluates to `false` for every possible value of `temp`.
- • To fix this bug, the programmer must replace `AND` with OR: `IF (temp < 2 OR temp > 8)`.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** An `ELSE` clause is completely optional in selection structures.
✕ **(C):** `DISPLAY` routinely outputs string literals enclosed in quotes in AP CSP pseudocode.
✕ **(D):** Relational operators `<` and `>` are standard tools for numeric comparisons.

Question 21 · Sequential IF Statements vs. Mutually Exclusive Branches

Priority ★★★★★

CORRECT ANSWER: (C) "D"

- Trace each independent `IF` statement in order:
- • Line 1: `score` is 95, `grade` is "F".
- • Check 1: `score >= 90` -> `95 >= 90` is `true`. `grade` becomes "A".
- • Check 2: Because these are independent `IF` statements (not `ELSE IF`), check 2 also executes! `score >= 80` -> `95 >= 80` is `true`. `grade` is overwritten to "B".
- • Check 3: `score >= 70` -> `95 >= 70` is `true`. `grade` is overwritten to "C".
- • Check 4: `score >= 60` -> `95 >= 60` is `true`. `grade` is overwritten to "D".
- • Therefore, the final value displayed is `"D"`.
- • (To fix this classic AP bug, the programmer must use chained `ELSE IF` statements or test from lowest to highest with appropriate bounds).

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Assumes none of the conditions execute and the initial default "F" remains.
✕ **(B):** Assumes the code stops at the second condition (`grade <- "B"`).
✕ **(D):** Assumes the program stops executing further checks once `score >= 90` is satisfied. Independent `IF` statements always test all subsequent conditions.

Question 22 · Nested Conditionals Decision Tree

Priority ★★★★

CORRECT ANSWER: (D) "Needs More Practice"

- Trace the nested decision branches step-by-step:
- • Outer check: `age >= 16 -> 16 >= 16` is `true`. Enter the outer `IF` block.
- • Middle check: `hasPermit = true -> true = true` is `true`. Enter the middle `IF` block.
- • Inner check: `hoursDriven >= 50 -> 45 >= 50` is `false`. Execute the inner `ELSE` block.
- • The inner `ELSE` block assigns: `status <- "Needs More Practice"`.
- • Therefore, `DISPLAY(status)` outputs `"Needs More Practice"`.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Requires `hoursDriven >= 50`, but `hoursDriven` is only 45.
✕ **(B):** Executes only if `hasPermit` is `false`.
✕ **(C):** Executes only if `age < 16`.

Question 23 · Equivalence of Nested IFs and Compound AND

Priority ★★★★★

CORRECT ANSWER: (A) Implementation 1 and Implementation 2 produce the exact same value for access for all combinations of Boolean inputs.

- Analyze the logical equivalence between nested `IF`s and logical conjunction (`AND`):
- • In Implementation 1, `access` is set to `true` if and only if `isEmployee = true` AND the inner block is reached where `hasBadge = true`.
- • If `isEmployee` is `false`, the inner block is never reached (`access` remains `false`).
- • If `hasBadge` is `false`, the inner assignment does not execute (`access` remains `false`).
- • In Implementation 2, the compound condition (`isEmployee = true AND hasBadge = true`) requires both conditions to be `true` simultaneously.
- • Both structures produce `true` under the exact same condition (`isEmployee = true` and `hasBadge = true`) and `false` otherwise.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(B):** If `hasBadge` is `false`, the inner `IF` test evaluates to `false`, so `access` remains `false` in Implementation 1.
✕ **(C):** The `AND` operator requires both conditions to be `true`, not just one.
✕ **(D):** Both evaluate the two conditions identically.

Question 24 · Maximum of Three Numbers Algorithm

Priority ★★★★

CORRECT ANSWER: (A) 27

- Trace the conditional branch execution:
- • Outer condition: `x > y -> 14 > 27` is `false`.
- • Jump to the outer `ELSE` block.
- • Inside outer `ELSE`: evaluate `y > z -> 27 > 19` is `true`.
- • Execute the inner `IF` branch: `maxVal <- y`.
- • Since `y` is 27, `maxVal` receives the value 27.
- • (This algorithm correctly determines the maximum among three numbers in all cases).

WHY OTHER CHOICES ARE INCORRECT:

✕ **(B):** 19 is `z`, which is smaller than `y` (27).
✕ **(C):** 14 is `x`, which is the minimum among the three numbers.
✕ **(D):** All paths in the decision tree assign a concrete value to `maxVal`.

Question 25 · Tiered Pricing & Nested Discounts

Priority ★★★★

CORRECT ANSWER: (B) 6

- Trace the pricing calculation step-by-step:
- • Line 1: `basePrice <- 12`.
- • Line 2: `age <- 68`.
- • First condition: `IF (age >= 65)` -> `68 >= 65` is `true`.
- - The `IF` block executes: `basePrice <- 8`.
- - The `ELSE` block is completely skipped.
- • Second independent check: `IF (isMatinee = true)` -> `true = true` is `true`.
- - The block executes: `basePrice <- basePrice - 2` -> `8 - 2 = 6`.
- • `DISPLAY(basePrice)` outputs 6.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Incorrectly applies both child and senior discounts.
✕ **(C):** Forgets to apply the matinee discount of $2 to the senior price of $8.
✕ **(D):** Applies the matinee discount to the standard adult price ($12 - $2 = $10), ignoring the senior discount.

Question 26 · Detecting Dead/Unreachable Code

Priority ★★★★★

CORRECT ANSWER: (C) The message "Target detected" will never be displayed for any integer value of val.

- Analyze the nested conditions for logical satisfiability:
- • To reach `DISPLAY("Target detected")`, the program must first satisfy the outer condition: `val > 50`.
- • Once inside, it must satisfy the inner condition: `val < 20`.
- • A number cannot be simultaneously greater than 50 and less than 20 (`val > 50 AND val < 20` is impossible).
- • Therefore, the inner condition is never `true`, making `DISPLAY("Target detected")` dead (unreachable) code.
- • For any `val > 50`, the inner `ELSE` branch will always execute, displaying "High range".

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** "Target detected" can never execute under any circumstances.
✕ **(B):** "Standard range" is displayed whenever `val <= 50` (e.g., `val = 10`).
✕ **(D):** "High range" is displayed whenever `val > 50` (e.g., `val = 60`).

Question 27 · Cumulative Score & Independent Conditionals

Priority ★★★★

CORRECT ANSWER: (C) 5

- Trace the accumulation of points step-by-step:
- • `points` starts at 0. `speed = 72`, `speedLimit = 55`.
- • First check: `speed > speedLimit` -> `72 > 55` is `true`. `points <- 0 + 2 = 2`.
- • Second check: `speed > speedLimit + 15` -> `72 > (55 + 15)` -> `72 > 70` is `true`.
- - `points <- points + 3` -> `2 + 3 = 5`.
- • Third check: `isSchoolZone = true` -> `false = true` is `false`. The school zone multiplier does not execute.
- • Therefore, `DISPLAY(points)` outputs 5.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Executes only the second check, assuming it overwrites or replaces the first penalty.
✕ **(B):** Multiplies by 2 even though `isSchoolZone` is `false`.
✕ **(D):** Fails to execute the second check for exceeding the speed limit by over 15 mph.

Question 28 · Converting Nested Conditionals to Single Expressions

Priority ★★★★

CORRECT ANSWER: (D) IF (hasPrerequisite = true AND NOT classFull) { canEnroll <- true }

- Analyze the conditions required to set `canEnroll` to `true`:
- • In the original code, `canEnroll` is set to `true` if and only if `hasPrerequisite = true` AND the inner check succeeds.
- • The inner check succeeds when `classFull = false`, which is equivalent to `NOT classFull`.
- • Combining both conditions with the `AND` operator gives: `(hasPrerequisite = true AND NOT classFull)`.
- • If either prerequisite is missing OR the class is full, the block does not execute and `canEnroll` remains `false`.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Tests the exact opposite condition.
✕ **(B):** Using `OR` allows enrollment if the class is not full even if the student lacks prerequisites.
✕ **(C):** Requires `classFull` to be `true`, allowing enrollment only into already-full classes.

Question 29 · Shipping Tier Boundary Error Debugging

Priority ★★★★★

CORRECT ANSWER: (A) 100

- Analyze the boundary condition at `orderTotal = 100`:
- • The store policy states: 'Purchases from $50 up to $100 pay $5 shipping.' This means an order of exactly $100 should pay $5.
- • Only purchases strictly 'over $100' should receive free shipping ($0).
- • In the code: When `orderTotal` is 100, `orderTotal < 50` is `false`. In the `ELSE` block, `orderTotal < 100` evaluates to `100 < 100`, which is `false`!
- • Because it is `false`, the code falls into `shipping <- 0` (free shipping).
- • Thus, for an order of exactly $100, the code charges $0 instead of $5, violating the policy.
- • (To fix the bug, the inner condition should be `orderTotal <= 100`).

WHY OTHER CHOICES ARE INCORRECT:

✕ **(B):** At 49.99: `49.99 < 50` is `true`; assigns `shipping <- 10` (correct according to policy).
✕ **(C):** At 50: `50 < 50` is `false`; inner `50 < 100` is `true`; assigns `shipping <- 5` (correct according to policy).
✕ **(D):** At 100.01: `100.01 < 100` is `false`; assigns `shipping <- 0` (correct according to policy).

Question 30 · Nested Conditional State Machine Trace

Priority ★★★★

CORRECT ANSWER: (B) 110

- Trace the conditional state changes:
- • `altitude = 100`, `command = "ASCEND"`, `fuelLevel = 15`.
- • Check outer `IF`: `command = "ASCEND"` is `true`. Enter the block.
- • Check inner `IF`: `fuelLevel > 20` -> `15 > 20` is `false`.
- • Execute the inner `ELSE` branch: `altitude <- altitude + 10` -> `100 + 10 = 110`.
- • The outer `ELSE` is skipped entirely.
- • `DISPLAY(altitude)` outputs `110`.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Assumes altitude remains unchanged at 100.
✕ **(C):** Executes the inner `IF` branch (`100 + 50 = 150`), which requires `fuelLevel > 20`.
✕ **(D):** Executes the `DESCEND` command (`100 - 20 = 80`).

Question 31 · REPEAT n TIMES Accumulator Trace

Priority ★★★★★

CORRECT ANSWER: (C) 10

- Trace each iteration of the loop:
- • Initial values: `count = 0`, `sum = 0`.
- • Iteration 1: `count <- 0 + 1 = 1`; `sum <- 0 + 1 = 1`.
- • Iteration 2: `count <- 1 + 1 = 2`; `sum <- 1 + 2 = 3`.
- • Iteration 3: `count <- 2 + 1 = 3`; `sum <- 3 + 3 = 6`.
- • Iteration 4: `count <- 3 + 1 = 4`; `sum <- 6 + 4 = 10`.
- • The loop finishes after 4 iterations.
- • `DISPLAY(sum)` outputs `10`.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Displays the final loop iteration count (`count = 4`) rather than the accumulated `sum`.
✕ **(B):** Stops one iteration early after 3 iterations (1 + 2 + 3 = 6).
✕ **(D):** Assumes `sum` adds 4 four times (`4 * 4 = 16`).

Question 32 · REPEAT UNTIL Termination Condition

Priority ★★★★★

CORRECT ANSWER: (C) 8

- Trace the loop execution and termination condition:
- • Initially: `n = 1`. Condition `n >= 8` is `1 >= 8` (false), so loop executes.
- • Iteration 1: `n <- 1 * 2 = 2`. Condition `2 >= 8` is `false`, so loop continues.
- • Iteration 2: `n <- 2 * 2 = 4`. Condition `4 >= 8` is `false`, so loop continues.
- • Iteration 3: `n <- 4 * 2 = 8`. Now test the condition: `8 >= 8` is `true`!
- • Because the termination condition evaluates to `true`, the loop stops immediately.
- • `DISPLAY(n)` outputs `8`.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Assumes `n` stops before reaching 8.
✕ **(B):** Terminates early after only two doublings.
✕ **(D):** Executes an unnecessary extra iteration (`8 * 2 = 16`), failing to stop as soon as `n >= 8` becomes true.

Question 33 · Loop Algorithm Equivalence

Priority ★★★★

CORRECT ANSWER: (A) sum <- 0 i <- 2 REPEAT 5 TIMES { sum <- sum + i i <- i + 2 }

- Verify Option A step-by-step:
- • `sum <- 0`, `i <- 2`.
- • There are 5 even numbers between 2 and 10 ({2, 4, 6, 8, 10}), so repeating 5 times is the correct count.
- • Iterations:
- - Iteration 1: `sum = 2`, `i` becomes 4.
- - Iteration 2: `sum = 6`, `i` becomes 6.
- - Iteration 3: `sum = 12`, `i` becomes 8.
- - Iteration 4: `sum = 20`, `i` becomes 10.
- - Iteration 5: `sum = 30`, `i` becomes 12.
- • The final `sum` is 30, exactly matching the requirement.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(B):** Terminates as soon as `i = 10`, meaning the value 10 is never added to `sum` (yields `sum = 20`, an off-by-one error).
✕ **(C):** Never increments `i`, creating an infinite accumulation of `1 * 2 = 2` for 5 iterations (`sum = 10`).
✕ **(D):** Repeats 10 times instead of 5, continuing up to `i = 20` and computing `sum = 110`.

Question 34 · Infinite Loop Identification

Priority ★★★★★

CORRECT ANSWER: (D) The code segment results in an infinite loop because i will skip over 10 and never equal 10.

- Trace the values of variable `i` across iterations:
- • Initially: `i = 1` (odd).
- • Iteration 1: `i` becomes 3.
- • Iteration 2: `i` becomes 5.
- • Iteration 3: `i` becomes 7.
- • Iteration 4: `i` becomes 9.
- • Iteration 5: `i` becomes 11.
- • Notice that `i` begins as an odd integer and is incremented by 2 on each step. It generates only odd numbers {1, 3, 5, 7, 9, 11, 13, ...}.
- • The target condition `i = 10` requires `i` to be an even number.
- • Because `i` skips directly from 9 to 11, `i = 10` is never satisfied, resulting in an infinite loop.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Assumes the loop stops at 9.
✕ **(B):** The syntax is valid AP CSP pseudocode; the error is a runtime logical infinite loop.
✕ **(C):** Assumes `i` stops at 10 after 5 iterations (`10 / 2 = 5`), ignoring that 10 is never reached.

Question 35 · Nested Iteration Execution Count

Priority ★★★★

CORRECT ANSWER: (B) 24

- Calculate the total executions of the inner loop body:
- • The outer loop repeats 3 times.
- • For each single iteration of the outer loop, the inner loop repeats 4 times.
- • Total number of inner loop executions = `3 × 4 = 12` times.
- • On each execution, `totalStars` increases by 2: `12 × 2 = 24`.
- • Therefore, `DISPLAY(totalStars)` outputs 24.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Calculates the number of loop iterations (12) without multiplying by the increment value (+2).
✕ **(C):** Adds 3 and 4 instead of multiplying (`(3 + 4) * 2 = 14`).
✕ **(D):** Multiplies `24 * 2 = 48`.

Question 36 · Loop Iteration Counting

Priority ★★★★

CORRECT ANSWER: (C) 4

- Trace the loop variables step-by-step:
- • Initial: `val = 25`, `steps = 0`. Condition `val < 5` is `false`.
- • Iteration 1: `val <- 25 - 6 = 19`; `steps <- 1`. Condition `19 < 5` is `false`.
- • Iteration 2: `val <- 19 - 6 = 13`; `steps <- 2`. Condition `13 < 5` is `false`.
- • Iteration 3: `val <- 13 - 6 = 7`; `steps <- 3`. Condition `7 < 5` is `false`.
- • Iteration 4: `val <- 7 - 6 = 1`; `steps <- 4`. Condition `1 < 5` is `true`!
- • Loop terminates because `val < 5` is now `true`.
- • `DISPLAY(steps)` outputs 4.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Counts 5 iterations, but after 4 iterations `val` is already 1, which terminates the loop.
✕ **(B):** Terminates early when `val` is 7, but 7 is not strictly less than 5.
✕ **(D):** Displays the initial value of `val` instead of `steps`.

Question 37 · Off-by-One Boundary Error in Iteration

Priority ★★★★★

CORRECT ANSWER: (A) It displays 1, 2, 3, 4 (stops before displaying 5).

- Trace the loop execution and DISPLAY calls:
- • Initially: `num = 1`. `num = 5` is `false`.
- - Displays 1, then `num` becomes 2.
- • Second time: `num = 2`. `num = 5` is `false`.
- - Displays 2, then `num` becomes 3.
- • Third time: `num = 3`. `num = 5` is `false`.
- - Displays 3, then `num` becomes 4.
- • Fourth time: `num = 4`. `num = 5` is `false`.
- - Displays 4, then `num` becomes 5.
- • Fifth check: Before executing the body, test condition: `num = 5` is `true`!
- • Because the condition is `true`, the loop terminates immediately WITHOUT displaying 5.
- • This is an off-by-one bug; to include 5, the condition should be `num > 5`.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(B):** Assumes 5 is displayed. The condition `num = 5` is evaluated before the loop body runs, terminating before `DISPLAY(5)`.
✕ **(C):** The loop terminates cleanly after 4 cycles.
✕ **(D):** Displays through 6 only if the loop termination condition was `num > 6`.

Question 38 · Selection Nested Within Iteration

Priority ★★★★

CORRECT ANSWER: (A) 3

- Trace the values of `i` through all 6 iterations:
- • `i` starts at 1 and increments by 1 on each iteration, testing values {1, 2, 3, 4, 5, 6}.
- • Check `i MOD 2 = 0` (even numbers):
- - `i = 1`: odd -> `evenCount` = 0
- - `i = 2`: even -> `evenCount` becomes 1
- - `i = 3`: odd -> `evenCount` = 1
- - `i = 4`: even -> `evenCount` becomes 2
- - `i = 5`: odd -> `evenCount` = 2
- - `i = 6`: even -> `evenCount` becomes 3
- • After 6 iterations, exactly 3 numbers {2, 4, 6} were even.
- • `DISPLAY(evenCount)` outputs 3.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(B):** Overcounts the evens.
✕ **(C):** Misses one of the even numbers.
✕ **(D):** Assumes every iteration increments `evenCount` (outputting 6).

Question 39 · Logarithmic Repeated Division Trace

Priority ★★★★★

CORRECT ANSWER: (B) 6

- Trace the division and step counter:
- • Initially: `num = 64`, `steps = 0`.
- • Iteration 1: `num <- 64 / 2 = 32`; `steps <- 1`.
- • Iteration 2: `num <- 32 / 2 = 16`; `steps <- 2`.
- • Iteration 3: `num <- 16 / 2 = 8`; `steps <- 3`.
- • Iteration 4: `num <- 8 / 2 = 4`; `steps <- 4`.
- • Iteration 5: `num <- 4 / 2 = 2`; `steps <- 5`.
- • Iteration 6: `num <- 2 / 2 = 1`; `steps <- 6`.
- • Termination check: `num <= 1` -> `1 <= 1` is `true`. Loop stops.
- • `DISPLAY(steps)` outputs 6 (since $2^6 = 64$).

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Performs an extra division to 0.5.
✕ **(C):** Outputs the original value of `num`.
✕ **(D):** Stops at `num = 2` (5 steps), but `2 <= 1` is `false`, so another halving occurs.

Question 40 · Trace Table with Interdependent Loop Updates

Priority ★★★★

CORRECT ANSWER: (D) 16 9

- Construct a variable state trace table across all 3 iterations of the loop:
- • Initial values before loop: `a = 2`, `b = 5`.
- • Iteration 1:
- - `a <- a + b` -> `2 + 5 = 7` (Variable a is updated to 7).
- - `b <- a - b` -> uses the NEW value of a: `7 - 5 = 2` (Variable b is updated to 2).
- - State after Iteration 1: `a = 7`, `b = 2`.
- • Iteration 2:
- - `a <- a + b` -> `7 + 2 = 9` (Variable a is updated to 9).
- - `b <- a - b` -> `9 - 2 = 7` (Variable b is updated to 7).
- - State after Iteration 2: `a = 9`, `b = 7`.
- • Iteration 3:
- - `a <- a + b` -> `9 + 7 = 16` (Variable a is updated to 16).
- - `b <- a - b` -> `16 - 7 = 9` (Variable b is updated to 9).
- - State after Iteration 3: `a = 16`, `b = 9`.
- • Final output: `DISPLAY(a)` and `DISPLAY(b)` output `16 9`.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Reflects the state after only 1 iteration (`a = 7, b = 2`).
✕ **(B):** Reflects the state if the loop had executed 4 iterations (`a = 16 + 9 = 25, b = 25 - 9 = 16`).
✕ **(C):** Reflects the state after only 2 iterations (`a = 9, b = 7`), stopping one cycle early.

Question 41 · Robot Grid Sequential Navigation

Priority ★★★★★

CORRECT ANSWER: (C) MOVE_FORWARD() MOVE_FORWARD() ROTATE_RIGHT() MOVE_FORWARD() MOVE_FORWARD()

- Trace the robot's movement step-by-step from (Row 4, Col 1) facing North:
- • `MOVE_FORWARD()`: moves 1 square North to (Row 3, Col 1).
- • `MOVE_FORWARD()`: moves 1 square North to (Row 2, Col 1).
- • `ROTATE_RIGHT()`: robot turns 90° clockwise in place; now facing East (▶).
- • `MOVE_FORWARD()`: moves 1 square East to (Row 2, Col 2).
- • `MOVE_FORWARD()`: moves 1 square East to (Row 2, Col 3).
- • The robot successfully arrives at the grey goal square [G] at (Row 2, Col 3).

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Turns East immediately from the starting square, ending at (Row 3, Col 3).
✕ **(B):** Moves North only 1 square, ending up at (Row 3, Col 3) rather than Row 2.
✕ **(D):** Moves North 3 squares to Row 1, overshooting the goal square.

Question 42 · Robot Iteration with REPEAT Loops

Priority ★★★★

CORRECT ANSWER: (A) Row 2, Column 5, facing East

- Trace the coordinates and orientation after each loop:
- • Start: (Row 5, Col 1) facing North (▲).
- • `REPEAT 3 TIMES { MOVE_FORWARD() }`:
- - Move 1: (Row 4, Col 1)
- - Move 2: (Row 3, Col 1)
- - Move 3: (Row 2, Col 1). Robot remains facing North.
- • `ROTATE_RIGHT()`: Robot turns 90° clockwise; now facing East (▶).
- • `REPEAT 4 TIMES { MOVE_FORWARD() }`:
- - Move 1: (Row 2, Col 2)
- - Move 2: (Row 2, Col 3)
- - Move 3: (Row 2, Col 4)
- - Move 4: (Row 2, Col 5).
- • Final position: Row 2, Column 5, facing East.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(B):** Assumes the robot moved 4 squares North and did not turn East.

Question 43 · Robot Obstacle Avoidance Around Black Squares

Priority ★★★★★

CORRECT ANSWER: (A) ROTATE_RIGHT() MOVE_FORWARD() ROTATE_LEFT() MOVE_FORWARD() MOVE_FORWARD() ROTATE_LEFT() MOVE_FORWARD()

- Trace Option A step-by-step from (Row 3, Col 2) facing North:
- • `ROTATE_RIGHT()`: faces East (▶).
- • `MOVE_FORWARD()`: moves to (Row 3, Col 3). Safe!
- • `ROTATE_LEFT()`: faces North (▲).
- • `MOVE_FORWARD()`: moves to (Row 2, Col 3). Safe bypass of [X]!
- • `MOVE_FORWARD()`: moves to (Row 1, Col 3). Safe!
- • `ROTATE_LEFT()`: faces West (◀).
- • `MOVE_FORWARD()`: moves to (Row 1, Col 2), which is the Goal [G]!
- • All movements avoid the black obstacle square [X] at (Row 2, Col 2).

WHY OTHER CHOICES ARE INCORRECT:

✕ **(B):** Immediately executes `MOVE_FORWARD()` North from (Row 3, Col 2), which crashes directly into the obstacle [X] at (Row 2, Col 2).
✕ **(C):** Moves North only 1 square to Row 2 before turning West, crashing into the obstacle from the East side.
✕ **(D):** Turns left to (Row 3, Col 1), then moves North to (Row 2, Col 1), but then turns right and moves into the obstacle [X] from the West.

Question 44 · CAN_MOVE Condition in Termination Loops

Priority ★★★★★

CORRECT ANSWER: (B) Row 1, Column 3, facing East

- Trace the loop until the wall boundary is reached:
- • Initial: (Row 4, Col 2) facing North.
- • Step 1: `CAN_MOVE(forward)` is `true`. `MOVE_FORWARD()` -> (Row 3, Col 2).
- • Step 2: `CAN_MOVE(forward)` is `true`. `MOVE_FORWARD()` -> (Row 2, Col 2).
- • Step 3: `CAN_MOVE(forward)` is `true`. `MOVE_FORWARD()` -> (Row 1, Col 2).
- • Step 4: At Row 1, the robot faces the outer northern grid wall. `CAN_MOVE(forward)` evaluates to `false`.
- • The condition `CAN_MOVE(forward) = false` is now `true`, terminating the loop.
- • After the loop:
- - `ROTATE_RIGHT()`: faces East (▶).
- - `MOVE_FORWARD()`: moves 1 square East to (Row 1, Col 3).
- • Final position: Row 1, Column 3, facing East.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Assumes the loop stopped before reaching the boundary wall at Row 1.
✕ **(C):** Turns left instead of right.
✕ **(D):** Omits the post-loop `ROTATE_RIGHT()` and `MOVE_FORWARD()` instructions.

Question 45 · Right-Hand Wall Following Algorithm

Priority ★★★★

CORRECT ANSWER: (B) It rotates left in place without moving forward.

- Trace the conditional branch execution:
- • Condition 1: `CAN_MOVE(right)` is `false`. The first `IF` body is skipped.
- • The code enters the outer `ELSE` block.
- • Condition 2: Inside `ELSE`, evaluate `CAN_MOVE(forward)`.
- • Because there is a dead-end wall ahead, `CAN_MOVE(forward)` is `false`.
- • The inner `IF` body is skipped, and execution falls into the inner `ELSE` block.
- • The inner `ELSE` executes `ROTATE_LEFT()`.
- • The robot simply rotates 90° to the left (now facing the open corridor) and terminates the step.
- • In the subsequent step, `CAN_MOVE(forward)` will be true, allowing it to move down the open corridor.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** `CAN_MOVE(right)` is false, so it does not turn right.
✕ **(C):** A single `ROTATE_LEFT()` is executed (90° turn), not a 180° reversal.
✕ **(D):** The code guards against collisions using `CAN_MOVE` checks; it never executes `MOVE_FORWARD()` when `CAN_MOVE(forward)` is false.

Question 46 · Algorithm Comparison for Grid Navigation

Priority ★★★★★

CORRECT ANSWER: (D) Both Algorithm 1 and Algorithm 2 successfully move the robot to (Row 5, Col 5).

- Analyze the movement of both algorithms:
- • Start: (Row 3, Col 3) facing South (▼).
- • Algorithm 1:
- - `MOVE_FORWARD()` twice South: (Row 4, Col 3) -> (Row 5, Col 3).
- - `ROTATE_LEFT()` from South turns to East (▶).
- - `MOVE_FORWARD()` twice East: (Row 5, Col 4) -> (Row 5, Col 5). Arrives at target!
- • Algorithm 2 (Staircase / Diagonal traversal):
- - Iteration 1: Move South to (Row 4, Col 3), Rotate Left to East, Move East to (Row 4, Col 4), Rotate Right to South.
- - Iteration 2: Move South to (Row 5, Col 4), Rotate Left to East, Move East to (Row 5, Col 5), Rotate Right to South.
- - Arrives at (Row 5, Col 5)!
- • Both algorithms arrive at the exact same target square (Row 5, Col 5).

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Algorithm 1 moves South, increasing row index towards Row 5, not Row 1.
✕ **(B):** Algorithm 2 takes a step-by-step zigzag path that remains fully within the 5x5 grid bounds without crashing.
✕ **(C):** Both algorithms successfully reach the destination.

Question 47 · Perimeter Traversal Loop Counting

Priority ★★★★

CORRECT ANSWER: (A) 12

- Calculate total executions of `MOVE_FORWARD()`:
- • The outer loop repeats 4 times (once for each of the 4 outer walls).
- • In each iteration, the inner loop calls `MOVE_FORWARD()` 3 times.
- • Total executions = `4 × 3 = 12` times.
- • Path verification: Row 1 Col 1 -> Col 4 (3 moves East), turns South -> Row 4 Col 4 (3 moves South), turns West -> Row 4 Col 1 (3 moves West), turns North -> Row 1 Col 1 (3 moves North).
- • The robot completes a full perimeter loop and returns to (Row 1, Col 1).

WHY OTHER CHOICES ARE INCORRECT:

✕ **(B):** Assumes a 4x4 grid requires 4 moves per edge (moving 4 times from Col 1 would attempt to move into Col 5, crashing into the outer wall).
✕ **(C):** Adds 4 and 3 instead of multiplying.
✕ **(D):** Counts only 3 edges of the perimeter.

Question 48 · Nested Robot Pattern Trace

Priority ★★★★

CORRECT ANSWER: (B) Row 1, Column 4

- Trace the 3 staircase iterations step-by-step:
- • Start: (Row 4, Col 1), facing East (▶).
- • Iteration 1:
- - Move East to (Row 4, Col 2).
- - Rotate Left to face North (▲).
- - Move North to (Row 3, Col 2).
- - Rotate Right to face East (▶).
- • Iteration 2:
- - Move East to (Row 3, Col 3).
- - Rotate Left to face North (▲).
- - Move North to (Row 2, Col 3).
- - Rotate Right to face East (▶).
- • Iteration 3:
- - Move East to (Row 2, Col 4).
- - Rotate Left to face North (▲).
- - Move North to (Row 1, Col 4).
- - Rotate Right to face East (▶).
- • The robot finishes at Row 1, Column 4.

WHY OTHER CHOICES ARE INCORRECT:

Question 49 · Robot Orientation Bug Diagnostics

Priority ★★★★★

CORRECT ANSWER: (B) The student used ROTATE_LEFT() instead of ROTATE_RIGHT(), causing the robot to face West instead of East.

- Diagnose the directional orientation error:
- • The robot started at Column 1 facing North.
- • After moving forward twice, the robot is at (Row 1, Col 1) facing North.
- • The target (Row 1, Col 3) is to the East (to the right of North).
- • Calling `ROTATE_LEFT()` caused the robot to rotate 90° counter-clockwise, facing West.
- • Since the robot was already in Column 1 (the leftmost column), calling `MOVE_FORWARD()` facing West attempted to move into Column 0, crashing into the boundary wall.
- • Replacing `ROTATE_LEFT()` with `ROTATE_RIGHT()` turns the robot East toward Column 3.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** `MOVE_FORWARD()` always moves in the direction the robot is currently facing.
✕ **(C):** Moving forward 3 times from Row 3 would crash into the North wall at Row 1.
✕ **(D):** Consecutive `MOVE_FORWARD()` commands are completely standard and valid.

Question 50 · Robot Infinite Oscillation Loop

Priority ★★★★★

CORRECT ANSWER: (C) The code results in an infinite loop because the robot oscillates in place without moving, so the wall ahead is never reached.

- Analyze the loop body and termination condition:
- • Termination condition: `CAN_MOVE(forward) = false` (true only when facing a wall).
- • In the open room, the robot starts with open space ahead, so `CAN_MOVE(forward)` is `true`.
- • Inside the loop:
- - `ROTATE_RIGHT()` turns the robot 90° clockwise.
- - `ROTATE_LEFT()` immediately turns the robot 90° counter-clockwise back to North.
- • Crucially, there is NO `MOVE_FORWARD()` command inside the loop!
- • The robot repeatedly rotates right and left while remaining in the exact same square, facing North.
- • Because its position and heading never change, `CAN_MOVE(forward)` remains `true` forever, creating an infinite loop.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** The robot never executes `MOVE_FORWARD()`, so it never moves toward the wall.
✕ **(B):** Both `ROTATE_RIGHT()` and `ROTATE_LEFT()` are valid commands; the issue is algorithmic stagnation.
✕ **(D):** Perimeter movement requires moving between squares.

Question 51 · 1-Based Indexing in College Board Pseudocode

Priority ★★★★★

CORRECT ANSWER: (C) red blue

- Apply 1-based indexing conventions of the AP CSP Reference Sheet:
- • `colors[1]` refers to the first element in the list, which is `"red"`.
- • `colors[2]` refers to the second element (`"green"`).
- • `colors[3]` refers to the third element (`"blue"`).
- • `colors[4]` refers to the fourth element (`"yellow"`).
- • Therefore, `DISPLAY(colors[1])` prints `red`, and `DISPLAY(colors[3])` prints `blue`.
- • Note: Unlike languages such as Python or Java that use 0-based indexing, AP CSP officially uses 1-based indexing for all lists and strings.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Outputs the first two elements instead of index 1 and index 3.
✕ **(B):** Assumes index 1 is "green" but index 3 is "blue".
✕ **(D):** Assumes 0-based indexing where index 1 would be "green" and index 3 would be "yellow".

Question 52 · APPEND and LENGTH Operations

Priority ★★★★★

CORRECT ANSWER: (D) 5 88

- Trace the list modifications:
- • Initial list: `scores` contains 3 elements: `[85, 92, 78]` (`LENGTH = 3`).
- • `APPEND(scores, 95)` adds 95 to the end: `[85, 92, 78, 95]` (`LENGTH = 4`).
- • `APPEND(scores, 88)` adds 88 to the end: `[85, 92, 78, 95, 88]` (`LENGTH = 5`).
- • `LENGTH(scores)` evaluates to 5.
- • `scores[LENGTH(scores)]` accesses the last element at index 5, which is 88.
- • The output displayed is `5 88`.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Miscounts the number of elements as 4.
✕ **(B):** Ignores the two APPEND operations, displaying the original length 3 and original last element 78.
✕ **(C):** Displays the second-to-last appended element (95) instead of the final element (88).

Question 53 · INSERT Operation & Element Shifting

Priority ★★★★★

CORRECT ANSWER: (D) C D

- Trace the list state before and after `INSERT`:
- • Initial: `letters = ["A", "B", "D", "E"]`.
- - Index 1: "A", Index 2: "B", Index 3: "D", Index 4: "E".
- • `INSERT(letters, 3, "C")` places "C" at index 3.
- - The old element at index 3 ("D") shifts right to index 4.
- - The element at index 4 ("E") shifts right to index 5.
- • New list: `["A", "B", "C", "D", "E"]`.
- • Index 3 is now `"C"`.
- • Index 4 is now `"D"`.
- • Therefore, `DISPLAY` outputs `C D`.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Assumes "D" was overwritten or deleted rather than shifted right.
✕ **(B):** Assumes 0-based indexing where index 3 would be "D".
✕ **(C):** Assumes "C" was appended to the end of the list without shifting.

Question 54 · REMOVE Operation & Left Shifting

Priority ★★★★★

CORRECT ANSWER: (B) eraser 3

- Trace the `REMOVE` operation:
- • Initial list: `items = ["pen", "notebook", "eraser", "ruler"]`.
- - Index 1: "pen", Index 2: "notebook", Index 3: "eraser", Index 4: "ruler".
- • `REMOVE(items, 2)` removes the element at index 2 (`"notebook"`).
- • All subsequent elements shift left by one position:
- - "eraser" moves from index 3 to index 2.
- - "ruler" moves from index 4 to index 3.
- • Updated list: `["pen", "eraser", "ruler"]`.
- • `items[2]` is now `"eraser"`.
- • `LENGTH(items)` is now 3.
- • Output displayed is `eraser 3`.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Displays index 1 ("pen") instead of index 2.
✕ **(C):** Assumes "notebook" remains at index 2 (i.e. that REMOVE did not execute).
✕ **(D):** Displays the new element at index 2 but forgets that `LENGTH` decreased from 4 to 3.

Question 55 · Interleaved List Insertion and Deletion

Priority ★★★★

CORRECT ANSWER: (C) "bird"

- Trace each list operation step-by-step:
- • Start: `["cat", "dog", "bird"]`.
- - Indices: [1:"cat", 2:"dog", 3:"bird"].
- • `INSERT(animals, 2, "fish")`:
- - "fish" is placed at index 2; "dog" and "bird" shift right.
- - List becomes: `["cat", "fish", "dog", "bird"]`.
- - Indices: [1:"cat", 2:"fish", 3:"dog", 4:"bird"].
- • `REMOVE(animals, 3)`:
- - The element at index 3 ("dog") is removed; "bird" shifts left to index 3.
- - List becomes: `["cat", "fish", "bird"]`.
- - Indices: [1:"cat", 2:"fish", 3:"bird"].
- • `APPEND(animals, "hamster")`:
- - Adds to the end: `["cat", "fish", "bird", "hamster"]`.
- - Indices: [1:"cat", 2:"fish", 3:"bird", 4:"hamster"].
- • `animals[3]` is `"bird"`.
- • `DISPLAY(animals[3])` outputs `"bird"`.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** "hamster" is at index 4 (the end of the list).
✕ **(B):** "dog" was removed from the list at step 2.
✕ **(D):** "fish" is at index 2, not index 3.

Question 56 · Swapping Elements in a List

Priority ★★★★★

CORRECT ANSWER: (C) temp <- nums[1] nums[1] <- nums[2] nums[2] <- temp

- Analyze the swap algorithm for list elements:
- • To swap `nums[1]` and `nums[2]` without losing data, one element must be temporarily saved in a variable `temp`.
- • Step 1: `temp <- nums[1]` saves 40.
- • Step 2: `nums[1] <- nums[2]` overwrites index 1 with 90.
- • Step 3: `nums[2] <- temp` overwrites index 2 with the saved 40.
- • Result: `nums` becomes `[90, 40, 60]`, successfully swapped.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** `temp` is used on line 1 before it has been initialized with any value.
✕ **(B):** Overwrites `nums[1]` with `nums[2]` on line 1, losing 40. Then `nums[2] <- nums[1]` sets index 2 to 90 as well, resulting in `[90, 90, 60]`.
✕ **(D):** Line 3 sets `nums[1] <- nums[2]` where `nums[2]` was already updated to 40, resulting in `[40, 40, 60]`.

Question 57 · Managing Program Complexity with Data Abstraction

Priority ★★★★★

CORRECT ANSWER: (A) Approach B allows algorithms (such as computing class average or finding the highest score) to be written concisely using loops, and easily accommodates changes in class size without rewriting code.

- Analyze data abstraction benefits per the AP CSP framework:
- • Data abstraction bundles related data items into a single unified structure (such as a list).
- • With Approach B, algorithms like computing the sum, average, maximum, or filtering can be expressed in a single loop (`FOR EACH score IN scores` or `REPEAT LENGTH(scores) TIMES`) rather than repeating code 30 times.
- • If the class size expands from 30 to 35 students, Approach B requires no changes to the algorithm (the loop automatically adapts to `LENGTH(scores)`), whereas Approach A would require declaring 5 new variables and modifying every calculation in the program.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(B):** Comparing list elements still requires relational and comparison operators.
✕ **(C):** Lists are stored in standard computer memory like any other data structure; they do not bypass hardware memory.
✕ **(D):** Lists organize data; they do not automatically prevent logic errors or runtime bugs.

Question 58 · Index Out of Bounds Error Analysis

Priority ★★★★

CORRECT ANSWER: (D) An index out-of-bounds error occurs because AP CSP lists are 1-based and 0 is not a valid index.

- Analyze list boundary constraints in the AP CSP exam reference:
- • The AP CSP exam explicitly defines lists as 1-indexed.
- • For a list of length N, the valid indices are strictly integers from 1 to N, inclusive.
- • Here, `roster` has length 3, so its valid indices are 1, 2, and 3.
- • Index 0 is outside the valid range (it is less than 1), resulting in an **index out-of-bounds error**.
- • (In contrast to languages like Python or C where index 0 is valid, in AP CSP pseudocode index 0 is always invalid).

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** "Alex" is located at index 1, not index 0.
✕ **(B):** Accessing an invalid index causes an out-of-bounds error rather than returning an empty string.
✕ **(C):** "Casey" is located at index 3.

Question 59 · Conditional Element Modification in Lists

Priority ★★★★

CORRECT ANSWER: (D) 24 18

- Trace each conditional element update:
- • Initially: `values = [12, 45, 8, 26]`.
- - `values[1] = 12`
- - `values[3] = 8`
- • Check 1: `values[1] < 20` -> `12 < 20` is `true`.
- - Execute body: `values[1] <- 12 * 2 = 24`.
- • Check 2: `values[3] < 10` -> `8 < 10` is `true`.
- - Execute body: `values[3] <- 8 + 10 = 18`.
- • Therefore, `DISPLAY(values[1])` outputs 24, and `DISPLAY(values[3])` outputs 18.
- • Final output: `24 18`.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Updates `values[1]` to 24 but fails to update `values[3]` to 18.
✕ **(B):** Updates `values[3]` to 18 but fails to update `values[1]` to 24.
✕ **(C):** Displays the un-modified original values 12 and 8, failing to execute the `IF` blocks.

Question 60 · List Assignment and Element Isolation

Priority ★★★★

CORRECT ANSWER: (C) 25 99

- Trace scalar variable assignment vs list element assignment:
- • Line 1: `listA` is initialized to `[10, 20, 30]`.
- • Line 2: `val <- listA[2]` copies the scalar numeric value 20 from index 2 into variable `val`.
- • Line 3: `val <- val + 5` updates `val` to `20 + 5 = 25`. This changes only `val`, leaving `listA[2]` unchanged at 20.
- • Line 4: `listA[2] <- 99` overwrites index 2 of `listA` with 99. This changes only the list element, leaving `val` unchanged at 25.
- • Line 5: `DISPLAY(val)` outputs 25.
- • Line 6: `DISPLAY(listA[2])` outputs 99.
- • Output is `25 99`.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Assumes `listA[2]` was not updated on Line 4.
✕ **(B):** Assumes `val` is a live pointer that changes when `listA[2]` is assigned 99.
✕ **(D):** Forgets to add 5 to `val` on Line 3.

Question 61 · FOR EACH List Traversal & Accumulation

Priority ★★★★★

CORRECT ANSWER: (C) 40

- Trace the `FOR EACH` loop execution through list elements:
- • `total` starts at 0.
- • Element 1: `item = 4` -> `total <- 0 + 4 = 4`.
- • Element 2: `item = 8` -> `total <- 4 + 8 = 12`.
- • Element 3: `item = 12` -> `total <- 12 + 12 = 24`.
- • Element 4: `item = 16` -> `total <- 24 + 16 = 40`.
- • All elements have been traversed.
- • `DISPLAY(total)` outputs `40`.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Stops before adding the final element (4 + 8 + 12 = 24).
✕ **(B):** Displays the last element of the list instead of the accumulated sum.
✕ **(D):** Displays the first element of the list.

Question 62 · Maximum Element Algorithm & Initializer Bug

Priority ★★★★★

CORRECT ANSWER: (B) [-15, -8, -42, -5]

- Analyze the initialization bug when all list values are negative:
- • In Option A: `numbers = [-15, -8, -42, -5]`. The true maximum is `-5`.
- • The student initializes `maxVal <- 0`.
- • In the loop: None of the values (-15, -8, -42, -5) are greater than 0 (`val > 0` is `false` for every item).
- • As a result, `maxVal` never updates and the algorithm outputs `0`.
- • Since 0 is not in the list and is greater than all list elements, the output is completely incorrect.
- • Correct implementation: Always initialize `maxVal <- numbers[1]` so that the initial baseline comes from the data itself.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** All values are positive; `maxVal` correctly updates to 25.
✕ **(C):** Contains a positive value (5); `maxVal` correctly updates to 5.
✕ **(D):** Contains positive values; `maxVal` correctly updates to 8.

Question 63 · Finding the Index of the Minimum Element

Priority ★★★★

CORRECT ANSWER: (B) 4 64

- Trace the minimum search algorithm with index tracking:
- • Initially: `minIndex = 1` (`scores[1] = 88`), `i = 2`.
- • `i = 2`: `scores[2] = 72 < 88` is `true`. `minIndex` becomes 2.
- • `i = 3`: `scores[3] = 95 < 72` is `false`. `minIndex` remains 2.
- • `i = 4`: `scores[4] = 64 < 72` is `true`. `minIndex` becomes 4.
- • `i = 5`: `scores[5] = 80 < 64` is `false`. `minIndex` remains 4.
- • Loop terminates because `i = 6 > 5`.
- • `DISPLAY(minIndex)` outputs 4.
- • `DISPLAY(scores[minIndex])` outputs `scores[4]`, which is 64.
- • Output is `4 64`.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Stops after comparing the first two elements, keeping index 2 (value 72).
✕ **(C):** Selects the last element examined.
✕ **(D):** Reverses the display order of the index and the value.

Question 64 · List Filtering Algorithm

Priority ★★★★★

CORRECT ANSWER: (D) aboveFreezing <- [] FOR EACH t IN temps { IF (t > 32) { APPEND(aboveFreezing, t) } }

- Analyze the filtering pattern in AP CSP pseudocode:
- • Step 1: Initialize an empty destination list: `aboveFreezing <- []`.
- • Step 2: Iterate over every element in the source list: `FOR EACH t IN temps`.
- • Step 3: Check if the element satisfies the criterion 'strictly greater than 32°F': `IF (t > 32)`.
- • Step 4: If `true`, append the item to the destination list: `APPEND(aboveFreezing, t)`.
- • This creates a clean, correctly filtered list containing only temperatures strictly above 32.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Cannot apply relational operator `>` directly to an entire list (`temps > 32` is a type error).
✕ **(B):** Modifies the list in-place while iterating over it, which skips elements due to index shifting, and attempts to pass an element value `t` into `REMOVE` instead of an integer index.
✕ **(C):** Uses inclusive inequality `>= 32`, which incorrectly includes 32°F (freezing point).

Question 65 · In-Place List Element Clamping

Priority ★★★★

CORRECT ANSWER: (D) [14, 0, 22, 0, 0]

- Trace the in-place replacement across all indices 1 through 5:
- • `i = 1`: `readings[1] = 14 >= 0` -> unchanged: 14.
- • `i = 2`: `readings[2] = -3 < 0` -> updated: `readings[2] <- 0`.
- • `i = 3`: `readings[3] = 22 >= 0` -> unchanged: 22.
- • `i = 4`: `readings[4] = -9 < 0` -> updated: `readings[4] <- 0`.
- • `i = 5`: `readings[5] = 0 >= 0` -> unchanged: 0.
- • The resulting list is `[14, 0, 22, 0, 0]`.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Assumes all numbers were overwritten with zero.
✕ **(B):** Assumes the list was not modified.
✕ **(C):** Replaces only the first negative number and fails to replace the second.

Question 66 · Target Element Occurrence Counter

Priority ★★★★

CORRECT ANSWER: (C) 3

- Count the occurrences of string `"YES"` in `votes`:
- • `v = "YES"` -> match #1: `yesCount` becomes 1.
- • `v = "NO"` -> no match: `yesCount` remains 1.
- • `v = "YES"` -> match #2: `yesCount` becomes 2.
- • `v = "YES"` -> match #3: `yesCount` becomes 3.
- • `v = "NO"` -> no match: `yesCount` remains 3.
- • `v = "ABSTAIN"` -> no match: `yesCount` remains 3.
- • Output displayed is 3.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Displays the total length of the list (6).
✕ **(B):** Counts the number of "NO" votes (2).
✕ **(D):** Counts only 1 match.

Question 67 · Index-Based Traversal vs. FOR EACH Traversal

Priority ★★★★★

CORRECT ANSWER: (B) When the algorithm needs to modify list elements at specific positions in place, or needs to access the adjacent element (such as comparing aList[i] with aList[i + 1]).

- Analyze the capabilities and limitations of `FOR EACH` vs index loops:
- • `FOR EACH item IN aList` provides read-only access to each element one at a time. It does not provide the numeric index position, nor does it allow looking ahead to adjacent elements (`aList[i + 1]`) or modifying list elements in-place.
- • If an algorithm needs to compare adjacent elements (e.g. checking if a list is sorted: `aList[i] > aList[i + 1]`) or assign new values to elements in-place (`aList[i] <- newVal`), an index counter `i` is required.
- • In contrast, calculating sums, counting occurrences, or testing element values can be done equally well using `FOR EACH`.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Calculating a sum can easily and concisely be accomplished with `FOR EACH item IN aList`.
✕ **(C):** Conditional counting can easily be accomplished with `FOR EACH`.
✕ **(D):** Boolean existence search does not require index tracking.

Question 68 · Comparing Two Lists for Exact Equality

Priority ★★★★

CORRECT ANSWER: (B) false

- Trace the list equality algorithm:
- • `LENGTH(list1) = 3` and `LENGTH(list2) = 3`. They have equal lengths, so the `ELSE` branch executes.
- • `i = 1`: `list1[1] = 10`, `list2[1] = 10`. They are equal (`10 ≠ 10` is `false`).
- • `i = 2`: `list1[2] = 20`, `list2[2] = 25`. They are NOT equal (`20 ≠ 25` is `true`).
- - The `IF` body executes: `areEqual <- false`.
- • `i = 3`: `list1[3] = 30`, `list2[3] = 30`. They are equal.
- • Because `areEqual` was set to `false` and there is no statement resetting it to `true`, `areEqual` remains `false`.
- • `DISPLAY(areEqual)` outputs `false`.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** The algorithm is syntactically and semantically valid.
✕ **(C):** Assumes `areEqual` reverts to `true` because index 3 has matching elements (30 = 30). There is no `ELSE { areEqual <- true }`, so the flag preserves the mismatch.
✕ **(D):** Boolean outputs are `true` or `false`.

Question 69 · Cumulative Running Total Generation

Priority ★★★★

CORRECT ANSWER: (A) [1, 3, 6, 10]

- Trace the running total accumulator and APPEND operations:
- • Start: `runningTotals = []`, `currentSum = 0`.
- • `val = 1`: `currentSum <- 0 + 1 = 1`. `APPEND(runningTotals, 1)` -> `[1]`.
- • `val = 2`: `currentSum <- 1 + 2 = 3`. `APPEND(runningTotals, 3)` -> `[1, 3]`.
- • `val = 3`: `currentSum <- 3 + 3 = 6`. `APPEND(runningTotals, 6)` -> `[1, 3, 6]`.
- • `val = 4`: `currentSum <- 6 + 4 = 10`. `APPEND(runningTotals, 10)` -> `[1, 3, 6, 10]`.
- • The resulting list contains `[1, 3, 6, 10]`.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(B):** Appends only the final total four times.
✕ **(C):** Appends `currentSum` before adding `val`.
✕ **(D):** Does not accumulate, simply copying the original list elements.

Question 70 · Concurrent Modification / Deletion Bug

Priority ★★★★★

CORRECT ANSWER: (D) When an element at index i is removed, the remaining elements shift left to fill the gap. Incrementing i immediately afterwards causes the algorithm to skip over the newly shifted element without checking it.

- Analyze the classic concurrent modification shifting bug:
- • Initially: `nums = [-5, -8, 12, 15]`. `i = 1`.
- • Iteration 1: `nums[1] = -5 < 0` is `true`. `REMOVE(nums, 1)` is called.
- - `-5` is removed! All subsequent elements shift left:
- - `-8` moves from index 2 to index 1!
- - `12` moves from index 3 to index 2.
- - `15` moves from index 4 to index 3.
- - New list: `[-8, 12, 15]`.
- - Then `i <- i + 1` increments `i` to 2!
- • Iteration 2: Now `i = 2`. The code checks `nums[2]`, which is `12`!
- • Notice that `-8` (now at index 1) was completely skipped and never checked!
- • Consequently, `-8` remains in the list, causing the algorithm to fail.
- • (To fix this bug, one should iterate backwards or only increment `i` when no deletion occurs).

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** `nums[i] < 0` is the standard check for negative numbers.
✕ **(B):** `REMOVE` works on any index regardless of element value.
✕ **(C):** In AP CSP pseudocode, `REPEAT n TIMES` evaluates `n` once at loop start.

Question 71 · Procedure Execution with Parameters and RETURN

Priority ★★★★★

CORRECT ANSWER: (D) 88

- Trace the procedure execution with the arguments `(100, 20, 10)`:
- • `basePrice = 100`, `discountPercent = 20`, `taxRate = 10`.
- • Step 1: `discounted <- 100 - (100 * 20 / 100)` -> `100 - 20 = 80`.
- • Step 2: `finalTotal <- 80 + (80 * 10 / 100)` -> `80 + 8 = 88`.
- • Step 3: `RETURN finalTotal` returns the value 88 to the caller.
- • `result` receives 88, and `DISPLAY(result)` outputs 88.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Computes a flat $10 tax on original $100 price (`80 + 10 = 90`) rather than calculating 10% on the discounted amount $80.
✕ **(B):** Calculates only the discounted price without adding tax ($80).
✕ **(C):** Returns the original base price without applying discount or tax.

Question 72 · Conditional Return in Procedures

Priority ★★★★

CORRECT ANSWER: (B) 42

- Trace the procedure execution with arguments `num1 = 35` and `num2 = 42`:
- • Evaluate condition: `num1 >= num2` -> `35 >= 42` is `false`.
- • The `IF` branch is skipped and the `ELSE` branch executes.
- • `RETURN num2` immediately returns 42 to the calling statement.
- • `val` is assigned 42, and `DISPLAY(val)` outputs 42.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Returns the smaller number (35), which would only happen if `num1 >= num2` were true.
✕ **(C):** Sums the two numbers (35 + 42 = 77).
✕ **(D):** Returns the Boolean evaluation of the comparison instead of the numeric value.

Question 73 · Benefits of Procedural Abstraction

Priority ★★★★★

CORRECT ANSWER: (B) It allows the damage calculation logic to be written once and reused across different game events, and allows developers to modify the damage formula in a single place without changing the rest of the game.

- Analyze the fundamental principles of procedural abstraction:
- • Procedural abstraction provides a named interface for a set of instructions, hiding implementation complexity from callers.
- • Benefits include:
- - Reusability: Any part of the game (e.g. traps, enemy hits, fall damage) calls `applyDamage` without duplicating logic.
- - Modularity & Maintainability: If the game designers decide to rebalance damage (e.g. adding armor reduction), they modify code inside `applyDamage` in one single location, and all calls throughout the game automatically use the updated formula.
- - Readability: Code becomes easier to understand and debug.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** All software requires rigorous testing regardless of whether procedures are used.
✕ **(C):** Procedural abstraction does not automatically alter asymptotic algorithmic time complexity.
✕ **(D):** Compilation to machine code is handled by compilers, not procedural abstraction.

Question 74 · Chained & Nested Procedure Calls

Priority ★★★★

CORRECT ANSWER: (A) 16

- Evaluate nested procedure calls from the inside out:
- • Inner call: `addFive(3)` is evaluated first.
- - Inside `addFive`: `RETURN 3 + 5` -> returns 8.
- • Outer call: Substitute the returned value 8 into the outer call: `doubleVal(8)`.
- - Inside `doubleVal`: `RETURN 8 * 2` -> returns 16.
- • `output` receives 16.
- • `DISPLAY(output)` prints 16.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(B):** Returns the intermediate value of the inner function (8) without executing `doubleVal`.
✕ **(C):** Evaluates in reverse order (`addFive(doubleVal(3)) = addFive(6) = 11`).
✕ **(D):** Adds rather than multiplying (`8 + 5 = 13`).

Question 75 · Parameters vs. Arguments Distinction

Priority ★★★★★

CORRECT ANSWER: (C) `width` and `height` are parameters; 7 and 12 are arguments.

- Distinguish between parameters and arguments:
- • **Parameters** are the variable placeholders defined in the procedure header that receive incoming data (here, `width` and `height`).
- • **Arguments** are the actual concrete values or expressions passed into the procedure during a specific call (here, `7` and `12`).
- • Therefore, `width` and `height` are parameters, and 7 and 12 are arguments.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** `computeArea` is the procedure name, and `rectArea` is the variable receiving the return value.
✕ **(B):** Mismatches variable names with numeric values.
✕ **(D):** Swaps the definitions of parameters and arguments.

Question 76 · Side Effects on Global Data Structures

Priority ★★★★

CORRECT ANSWER: (D) 4 Carlos

- Trace the procedure calls and side effects on the list `names`:
- • Initially: `names = ["Ana", "Ben"]` (length 2).
- • Call 1: `addStudent("Carlos")` appends "Carlos" to `names`.
- - `names` becomes `["Ana", "Ben", "Carlos"]` (length 3).
- • Call 2: `addStudent("Diana")` appends "Diana" to `names`.
- - `names` becomes `["Ana", "Ben", "Carlos", "Diana"]` (length 4).
- • `LENGTH(names)` evaluates to 4.
- • `names[3]` accesses the element at index 3, which is `"Carlos"`.
- • The output is `4 Carlos`.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Assumes modifications inside the procedure did not persist in the global list.
✕ **(B):** Displays the 4th element ("Diana") instead of the 3rd element ("Carlos").
✕ **(C):** Displays the initial length and 2nd element.

Question 77 · Local Variable Scope vs. Global Scope

Priority ★★★★★

CORRECT ANSWER: (D) 70 100

- Analyze variable scope and parameter shadowing:
- • Global variable `x` is initialized to 100.
- • The procedure `modifyVal` defines a parameter named `x`. Inside `modifyVal`, the parameter `x` is local to the procedure.
- • During the call `modifyVal(20)`:
- - Local `x` receives 20.
- - `x <- x + 50` updates local `x` to `20 + 50 = 70`.
- - `DISPLAY(x)` inside the procedure prints `70`.
- • When the procedure finishes, its local scope is destroyed.
- • The outer `DISPLAY(x)` accesses the global variable `x`, which remained untouched at 100.
- • Therefore, the output is `70 100`.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Assumes the local assignment overwrote the global variable outside the procedure.
✕ **(B):** Displays `x` before the addition of 50.
✕ **(C):** Passes the global value 100 into the procedure instead of the argument 20 (`100 + 50 = 150`).

Question 78 · Early Return in List Search Procedures

Priority ★★★★★

CORRECT ANSWER: (A) true; it stops executing immediately when 45 is encountered on the second iteration.

- Analyze the control flow of `RETURN` statements:
- • In AP CSP pseudocode and general computer science, when a `RETURN` statement executes, the procedure terminates **immediately**, returning control and the specified value to the caller.
- • In `containsValue(myList, 45)`:
- - Iteration 1: `item = 12 ≠ 45`. Loop continues.
- - Iteration 2: `item = 45 = 45` is `true`. `RETURN true` executes.
- • The procedure immediately terminates and returns `true`. It does NOT continue examining 78 or 92.
- • `result` receives `true`, and `DISPLAY(result)` prints `true`.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(B):** The loop does not reach `RETURN false` because the early return already exited the procedure.
✕ **(C):** Assumes the loop must complete all iterations before a `RETURN` can fire.
✕ **(D):** The procedure returns a Boolean (`true`), not the numeric target.

Question 79 · Generalizing Algorithms with Parameters

Priority ★★★★

CORRECT ANSWER: (A) PROCEDURE calcDues(baseAmount, discountRate) { RETURN baseAmount * (1 - discountRate) }

- Analyze procedural generalization and parameterization:
- • All three original procedures perform the exact same underlying mathematical task: applying a percentage discount to a base fee.
- • By introducing parameters for the variable factors (`baseAmount` and `discountRate`), a single procedure `calcDues(baseAmount, discountRate)` can handle freshmen, sophomores, seniors, and any future grade levels or fee adjustments.
- • This reduces redundant code, minimizes bugs, and exemplifies effective procedural abstraction.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(B):** Displays the amount without computing a discount.
✕ **(C):** Hardcodes the freshman discount and takes no parameters.
✕ **(D):** Always returns 50 regardless of grade level.

Question 80 · Chained Function Evaluation & Recursion Trace

Priority ★★★★★

CORRECT ANSWER: (C) 10

- Trace the recursive procedure calls step-by-step:
- • Call 1: `sumDown(4)` -> `4 > 1`, returns `4 + sumDown(3)`.
- • Call 2: `sumDown(3)` -> `3 > 1`, returns `3 + sumDown(2)`.
- • Call 3: `sumDown(2)` -> `2 > 1`, returns `2 + sumDown(1)`.
- • Call 4 (Base Case): `sumDown(1)` -> `1 <= 1` is `true`, returns `1`.
- • Unwind the call stack:
- - `sumDown(2) = 2 + 1 = 3`
- - `sumDown(3) = 3 + 3 = 6`
- - `sumDown(4) = 4 + 6 = 10`
- • `ans` receives 10, and `DISPLAY(ans)` outputs `10`.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Returns only the base case value 1.
✕ **(B):** Computes the factorial (product: 4 × 3 × 2 × 1 = 24) instead of the sum.
✕ **(D):** Returns only the input argument 4.

Question 81 · Binary Search Preconditions & Limitations

Priority ★★★★★

CORRECT ANSWER: (B) The elements in the list must already be sorted in ascending or descending order.

- Analyze the fundamental mechanics of binary search:
- • Binary search is a divide-and-conquer algorithm that compares the target value to the element at the middle index of the list.
- • If the target is smaller than the middle element, it eliminates the entire upper half of the list; if larger, it eliminates the lower half.
- • This elimination logic relies entirely on the mathematical guarantee that all elements prior to the middle are smaller and all elements after are larger.
- • Therefore, the list **MUST BE SORTED** beforehand. If the list is unsorted, binary search fails completely and will often fail to find items that are actually present.
- • Linear search, in contrast, checks elements sequentially from start to finish and works on both sorted and unsorted lists.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Binary search works on any data type with a well-defined ordering (such as alphabetical strings).
✕ **(C):** Binary search works on lists of any size, odd or even.
✕ **(D):** Binary search can operate on lists containing duplicates (it simply finds one instance of the duplicate).

Question 82 · Binary Search Worst-Case Comparisons

Priority ★★★★★

CORRECT ANSWER: (B) 11

- Calculate binary search worst-case efficiency:
- • Binary search halves the remaining search space with each comparison.
- • The number of elements remaining after k comparisons is approximately $N / 2^k$.
- • For $N = 1,024$:
- - Comparison 1: 1,024 -> 512 elements left
- - Comparison 2: 512 -> 256 elements left
- - Comparison 3: 256 -> 128 elements left
- - Comparison 4: 128 -> 64 elements left
- - Comparison 5: 64 -> 32 elements left
- - Comparison 6: 32 -> 16 elements left
- - Comparison 7: 16 -> 8 elements left
- - Comparison 8: 8 -> 4 elements left
- - Comparison 9: 4 -> 2 elements left
- - Comparison 10: 2 -> 1 element left
- - Comparison 11: 1 element compared; concludes target is found or absent.
- • The maximum number of comparisons is $\lfloor \log_2(1024) \rfloor + 1 = 10 + 1 = 11$ comparisons.
- • (In contrast, linear search in the worst case would require all 1,024 comparisons).

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** 100 is an arbitrary number.
✕ **(C):** 512 is the number of elements eliminated after just one comparison.
✕ **(D):** 1,024 is the worst-case number of comparisons for a linear search, not binary search.

Question 83 · Step-by-Step Binary Search Trace

Priority ★★★★

CORRECT ANSWER: (D) 33, then 55, then 47

- Trace the binary search steps on `[12, 19, 24, 33, 47, 55, 68]` with `target = 47`:
- • Step 1: `low = 1`, `high = 7`. Middle index = `(1 + 7) / 2 = 4`.
- - Compare target (47) with `data[4]` (value 33).
- - Since `47 > 33`, eliminate the lower half. Set `low = 4 + 1 = 5`.
- • Step 2: `low = 5`, `high = 7`. Middle index = `(5 + 7) / 2 = 6`.
- - Compare target (47) with `data[6]` (value 55).
- - Since `47 < 55`, eliminate the upper half. Set `high = 6 - 1 = 5`.
- • Step 3: `low = 5`, `high = 5`. Middle index = `(5 + 5) / 2 = 5`.
- - Compare target (47) with `data[5]` (value 47).
- - `47 = 47` -> Match found!
- • The sequence of values compared is `33, then 55, then 47`.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Calculates the initial middle index incorrectly.
✕ **(B):** Misses the comparison with 55 at middle index 6.
✕ **(C):** This is the sequence of comparisons for a linear search checking every element from left to right.

Question 84 · Random Number Range & Simulation Bounds

Priority ★★★★★

CORRECT ANSWER: (A) Minimum is 2, maximum is 12

- Evaluate the mathematical range of the random sum:
- • `die1` generates integers in the inclusive range [1, 6].
- • `die2` generates integers in the inclusive range [1, 6].
- • Minimum possible sum: `die1 = 1` and `die2 = 1` -> `1 + 1 = 2`.
- • Maximum possible sum: `die1 = 6` and `die2 = 6` -> `6 + 6 = 12`.
- • Therefore, `diceSum` can range from 2 to 12, inclusive.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(B):** Sets the maximum to 6, which is the maximum of a single die.
✕ **(C):** Assumes one die could produce 0, but the minimum for each die is 1 (so minimum sum is 2).

Question 85 · Simulating Probabilistic Real-World Events

Priority ★★★★

CORRECT ANSWER: (C) headsCount <- 0 REPEAT 1000 TIMES { toss <- RANDOM(1, 2) IF (toss = 1) { headsCount <- headsCount + 1 } }

- Analyze the simulation structure per AP CSP guidelines:
- • A fair coin has 2 equally likely outcomes (50% probability each).
- • `RANDOM(1, 2)` produces either 1 or 2 with equal 50% probability.
- • Testing `IF (toss = 1)` correctly identifies each heads outcome.
- • Placing this inside `REPEAT 1000 TIMES` executes exactly 1,000 independent trials.
- • `headsCount` correctly records the cumulative number of heads.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** `RANDOM(0, 2)` generates 3 possible outcomes ({0, 1, 2}), giving heads a 1/3 (33.3%) probability instead of 50%.
✕ **(B):** Simply picks a single random number between 1 and 1000 without simulating individual coin tosses.
✕ **(D):** Loops until `headsCount` reaches 1000, which does not simulate 1000 tosses.

Question 86 · Reasonable vs. Unreasonable Algorithmic Time

Priority ★★★★★

CORRECT ANSWER: (B) Exponential time: 2^n

- Classify algorithmic efficiency according to the College Board AP CSP curriculum:
- • **Reasonable time** means the algorithm runs in polynomial time or faster:
- - Constant: $O(1)$
- - Logarithmic: $O(\log n)$
- - Linear: $O(n)$
- - Quadratic: $O(n^2)$
- - Polynomial: $O(n^k)$ for any constant k.
- • **Unreasonable time** refers to algorithms whose steps grow exponentially or factorially:
- - Exponential time: $O(2^n)$
- - Factorial time: $O(n!)$
- • For unreasonable algorithms, even modest inputs (e.g., $n = 100$) take longer than the age of the universe to compute, making exact solutions intractable.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Linear time n is polynomial (n^1) and is considered reasonable time.
✕ **(C):** Logarithmic time $\log n$ is exceptionally fast and is considered reasonable time.
✕ **(D):** Quadratic time n^2 is polynomial and is considered reasonable time.

Question 87 · Heuristic Solutions for Optimization Problems

Priority ★★★★★

CORRECT ANSWER: (B) It provides a fast, approximate solution that is 'good enough' within reasonable time, even though it is not guaranteed to find the absolute mathematically optimal route.

- Define heuristic algorithms according to the AP CSP framework:
- • When an optimization problem takes unreasonable time (such as exponential 2^n or factorial $n!$), finding the exact perfect solution is computationally intractable for large inputs.
- • A **heuristic** is an algorithmic technique (such as 'always visit the nearest unvisited city next') designed to find a practical, approximate solution in reasonable polynomial time.
- • Trade-off: The heuristic does not guarantee the 100% absolute optimal route, but it finds a route that is extremely close to optimal in milliseconds rather than billions of years.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Heuristics are standard software algorithms executed on conventional everyday computers.
✕ **(C):** Heuristics do not make problems undecidable; the problem remains decidable.
✕ **(D):** Heuristics do not guarantee the absolute optimal answer, nor do they run in constant O(1) time.

Question 88 · Optimization Problems vs. Decision Problems

Priority ★★★★

CORRECT ANSWER: (A) Finding the lowest cost flight itinerary between New York and London among all available airlines.

- Distinguish between decision and optimization problems:
- • A **Decision Problem** is a computational problem that has a binary **yes-or-no** answer (e.g., 'Is there a flight under $500?', 'Is n prime?').
- • An **Optimization Problem** requires finding the **best** solution among all possible feasible solutions (e.g., minimizing cost, maximizing efficiency, finding the shortest path).
- • Finding the lowest cost flight itinerary searches for the minimum cost among all flights, making it an optimization problem.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(B):** This has a yes/no answer ('Is n prime?'), making it a decision problem.
✕ **(C):** This has a yes/no answer ('Does it have >= 8 characters?'), making it a decision problem.
✕ **(D):** This has a yes/no answer ('Does a flight under $500 exist?'), making it a decision problem.

Question 89 · Undecidable Problems & The Halting Problem

Priority ★★★★★

CORRECT ANSWER: (D) A problem for which it is mathematically impossible to construct an algorithm that will always produce a correct yes-or-no answer for every possible input.

- Analyze algorithmic decidability per the College Board curriculum:
- • An **undecidable problem** is a decision problem for which no algorithm can ever be written that is guaranteed to always provide a correct true/false answer for all possible valid inputs.
- • The **Halting Problem** asks: Can we write a general program H that takes any arbitrary program P and input I, and correctly determines whether P will eventually halt (finish) or run in an infinite loop?
- • Turing proved mathematically that no such program H can ever exist.
- • Crucially, an undecidable problem cannot be solved by building faster processors or adding more memory; it is fundamentally impossible in computational theory.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Undecidable problems can never be solved by faster computers; their unsolvability is mathematically proven.
✕ **(B):** Memory limitations are physical hardware constraints, not theoretical decidability limits.
✕ **(C):** Execution time relates to algorithmic efficiency (tractability), not decidability.

Question 90 · Computational Simulations & Real-World Modeling

Priority ★★★★★

CORRECT ANSWER: (C) It allows planners to test extreme scenarios (such as blizzards or severe accidents) safely and rapidly without creating real-world traffic gridlock, risking collisions, or incurring high physical costs.

- Analyze the benefits and limitations of computer simulations per AP CSP:
- • Advantages:
- - Safety: Planners can test hazardous conditions (ice, heavy congestion, emergency vehicle preemption) without endangering drivers.
- - Cost & Speed: Planners can run weeks of simulated traffic data in seconds without purchasing hardware or closing roads.
- - Exploration: Planners can rapidly iterate through hundreds of different timing configurations.
- • Limitations:
- - Simulations are computational models that inevitably require **simplifying assumptions** (e.g., driver reaction times, vehicle acceleration).
- - Therefore, simulations approximate reality but are not 100% perfect.

WHY OTHER CHOICES ARE INCORRECT:

✕ **(A):** Simulations always rely on abstractions and simplifying assumptions; they never match reality with 100% perfection.
✕ **(B):** Simulations model traffic to optimize signal timing; they do not eliminate the need for physical signals.
✕ **(D):** All computer simulations must make simplifying assumptions to be computationally feasible.

Question 91 · Model Solution & Scoring Rubric

School Cafeteria Account & Low-Balance Alert System ❖ Total Available Points: 9 Points

COMPREHENSIVE STEP-BY-STEP RESPONSE:

• **(a) Data Abstraction & Complexity Management:**
- The data abstraction is the list `accountBalances`, which stores floating-point decimal numbers representing dollar balances for enrolled students.
- *Managing Complexity:* Without a list, the program would require declaring hundreds of individual variables (`balance1`, `balance2`, `balance3`, etc.). Processing transactions would require massive, repetitive `IF-ELSE` chains. The list enables dynamic positional indexing (`accountList[studentIndex]`) that scales effortlessly to any school population without changing the algorithm.

• **(b) Procedural Abstraction & Algorithm Implementation:**
- *Procedure Name & Parameters:* `processTransaction(accountList, studentIndex, chargeAmount)`.
- *Algorithm Description:*
1. **Sequencing & Guard Check:** Evaluates whether `studentIndex` is within valid list bounds (between 1 and `LENGTH(accountList)`).
2. **Selection (Funds Check):** Checks `IF (currentBal >= chargeAmount)`. If true, subtracts `chargeAmount` from the student's balance in-place.
3. **Nested Selection (Alert):** If the updated balance is strictly below $5.00, calls `sendLowBalanceAlert(studentIndex)`. Returns `true`.
4. **Alternative Branch (Decline):** If funds are insufficient, calls `logDeclinedCharge` and returns `false` without modifying account balances.

• **(c) Testing with Different Call Scenarios:**
- **Call 1 (Sufficient Funds):** `processTransaction(accountBalances, 1, 10.00)`.
• Condition tested: `45.50 >= 10.00` evaluates to `true`.
• Result: `accountBalances[1]` updates to $35.50, procedure returns `true`.
- **Call 2 (Insufficient Funds):** `processTransaction(accountBalances, 3, 10.00)`.
• Condition tested: `3.25 >= 10.00` evaluates to `false`.
• Result: `logDeclinedCharge(3, 10.00)` is triggered, balance remains $3.25, procedure returns `false`.

• **(d) Index Bounds Safety:**
- In AP CSP 1-indexed lists, attempting to access index 0, negative indices, or an index greater than list length crashes the program with an 'index out of bounds' error.
- Defensive Guard: The outer check `IF (studentIndex >= 1 AND studentIndex <= LENGTH(accountList))` validates the index before any list access occurs, immediately returning `false` if an invalid student ID is supplied.

OFFICIAL COLLEGE BOARD 9-POINT SCORING RUBRIC:

Criteria / Evaluated Performance Skill	Points Allocated
(a) Correctly identifies `accountBalances` list storing numeric decimal balances, and explains that it manages complexity by allowing dynamic indexing and accommodating any number of students without rewriting code.	**2 pts**
(b) Identifies `processTransaction` with parameters `(accountList, studentIndex, chargeAmount)`, and explains algorithm: validates index bounds, checks sufficient funds, deducts charge, triggers low-balance alert if below $5.00, logs declines, and returns boolean status.	**3 pts**
(c) Call 1: `processTransaction(accountBalances, 1, 10.00)` -> executes successful deduction branch ($45.50 - $10.00 = $35.50, returns true). Call 2: `processTransaction(accountBalances, 3, 10.00)` -> executes insufficient funds branch ($3.25 < $10.00, logs decline, returns false).	**2 pts**
(d) Explains out-of-bounds error occurs if `studentIndex` is less than 1 or exceeds `LENGTH(accountList)`, and explains how `IF (studentIndex >= 1 AND studentIndex <= LENGTH(accountList))` guards against invalid memory access.	**2 pts**
TOTAL SCORING MAXIMUM	**9 / 9 Points**

Question 92 · Model Solution & Scoring Rubric

Fitness Tracker Step Goal Analyzer ❖ Total Available Points: 9 Points

COMPREHENSIVE STEP-BY-STEP RESPONSE:

• **(a) Data Abstraction & Scalability:**
- The data abstraction is `weeklySteps`, a 1-indexed list containing integer values representing daily step totals.
- *Scalability:* Because the algorithm uses `FOR EACH` and `LENGTH(stepsList)` dynamically, expanding the tracking duration to 30 days requires zero code modifications. The loop seamlessly traverses 30 elements and divides by 30.

• **(b) Algorithm Execution Details:**
- *Initialization:* Sets accumulators `totalSteps <- 0` and `daysGoalMet <- 0`.
- *Iteration:* Loops through each step count `dayCount` in `stepsList`.
- *Accumulation & Selection:* Adds `dayCount` to `totalSteps`. Concurrently checks `IF (dayCount >= dailyGoal)` and increments `daysGoalMet`.
- *Post-Loop Decision:* Computes `avgSteps <- totalSteps / LENGTH(stepsList)`. Evaluates compound condition `(daysGoalMet >= 5 AND avgSteps >= dailyGoal)` to determine achievement message.

• **(c) Trace with dailyGoal = 10,000:**
- Sum: 8,400 + 10,200 + 9,500 + 6,100 + 11,400 + 12,000 + 7,800 = **65,400 total steps**.
- Days >= 10,000: Day 2 (10,200), Day 5 (11,400), Day 6 (12,000) = **3 days**.
- Average: 65,400 / 7 = **9,342.8 steps/day**.
- Condition check: `(3 >= 5 AND 9342.8 >= 10000)` -> `false AND false` = `false`.
- Return value: **"Goal Incomplete. Keep pushing next week!"**.

• **(d) Call Scenarios & Branch Execution:**
- *Call 1:* `evaluateWeeklyProgress(weeklySteps, 8000)`. Days meeting 8,000 steps are 5 (8400, 10200, 9500, 11400, 12000). Average 9,342.8 >= 8,000 is true. Returns `"Goal Achieved! Outstanding consistency."`
- *Call 2:* `evaluateWeeklyProgress(weeklySteps, 12000)`. Days meeting 12,000 is only 1. Condition fails. Returns `"Goal Incomplete. Keep pushing next week!"`.

OFFICIAL COLLEGE BOARD 9-POINT SCORING RUBRIC:

Criteria / Evaluated Performance Skill	Points Allocated
(a) Identifies `weeklySteps` storing integer daily step counts, explains list data abstraction, and notes that 30 days would automatically be handled by `LENGTH(stepsList)` without changing the algorithm.	**2 pts**
(b) Details algorithm: initializes accumulators, iterates with `FOR EACH`, updates `totalSteps`, uses `IF` to count goal days, calculates average, and uses compound Boolean `IF (daysGoalMet >= 5 AND avgSteps >= dailyGoal)` for final status.	**3 pts**
(c) Correctly calculates: `totalSteps = 65,400`, `daysGoalMet = 3` (days with 10200, 11400, 12000), `avgSteps = 65400 / 7 ≈ 9342.8`, returning 'Goal Incomplete. Keep pushing next week!'.	**2 pts**
(d) Call 1: `evaluateWeeklyProgress(weeklySteps, 8000)` -> `daysGoalMet = 5`, `avg = 9342.8 >= 8000` -> returns 'Goal Achieved!'. Call 2: `evaluateWeeklyProgress(weeklySteps, 10000)` -> returns 'Goal Incomplete.'.	**2 pts**
TOTAL SCORING MAXIMUM	**9 / 9 Points**

Question 93 · Model Solution & Scoring Rubric

Warehouse Inventory Restock Prioritization Algorithm ❖ Total Available Points: 9 Points

COMPREHENSIVE STEP-BY-STEP RESPONSE:

• **(a) Parallel Lists Data Abstraction:**
- Parallel lists represent related attributes of the same physical item across multiple synchronized lists.
- Index correspondence: For any index `i`, `stockLevels[i]` and `reorderLimits[i]` refer to the exact same inventory item.
- This abstracts multi-field data records without requiring complex object structures.

• **(b) Algorithm Trace & Urgency Filtering:**
- *Iteration:* Loops through every index from `i = 1` to `LENGTH(stock)`.
- *Stock Deficit Check:* Evaluates `IF (currentStock < threshold)`. If stock is below minimum threshold, computes `deficit <- threshold - currentStock`.
- *Urgency Threshold:* Evaluates nested check `IF (deficit >= urgencyFactor)`. If the shortfall is severe enough, `APPEND(urgentOrders, i)` stores the item's index.
- *Return:* Returns the accumulated list of critical item indices.

• **(c) Execution with urgencyFactor = 7:**
- *Index 1:* stock 14, limit 20 -> deficit = 20 - 14 = 6. Condition `6 >= 7` is false.
- *Index 2:* stock 5, limit 15 -> deficit = 15 - 5 = 10. Condition `10 >= 7` is true! **Append 2**.
- *Index 3:* stock 42, limit 30 -> stock exceeds threshold. No action.
- *Index 4:* stock 2, limit 10 -> deficit = 10 - 2 = 8. Condition `8 >= 7` is true! **Append 4**.
- *Index 5:* stock 80, limit 50 -> stock exceeds threshold. No action.
- Returned result: **`[2, 4]`**.

• **(d) Infinite Loop Bug Diagnostic:**
- If `i <- i + 1` were moved inside `IF (deficit >= urgencyFactor)`, then whenever an item has no deficit (like Index 1 where deficit 6 < 7), `i` would never be incremented.
- On the next loop cycle, the code would re-evaluate Index 1 indefinitely, trapping the program in an **infinite loop**.

OFFICIAL COLLEGE BOARD 9-POINT SCORING RUBRIC:

Criteria / Evaluated Performance Skill	Points Allocated
(a) Explains that parallel lists use matching indices to link related attributes (index 1 of stock matches index 1 of limits), functioning as an aggregate data abstraction.	**2 pts**
(b) Explains algorithm: iterates from index 1 to `LENGTH(stock)`, compares stock to limit, calculates deficit, checks if deficit meets or exceeds `urgencyFactor`, and appends index `i` to `urgentOrders`.	**3 pts**
(c) Shows evaluation: Index 1 (deficit 6 < 7), Index 2 (deficit 10 >= 7 -> append 2), Index 3 (stock 42 > 30 -> no deficit), Index 4 (deficit 8 >= 7 -> append 4), Index 5 (stock 80 > 50 -> no deficit). Returns `[2, 4]`.	**2 pts**
(d) Explains that placing `i <- i + 1` inside the conditional block creates an infinite loop if an item's deficit does not meet the urgency factor, because `i` would never increment.	**2 pts**
TOTAL SCORING MAXIMUM	**9 / 9 Points**

Question 94 · Model Solution & Scoring Rubric

Autonomous Delivery Drone Route Safety Verification ❖ Total Available Points: 9 Points

COMPREHENSIVE STEP-BY-STEP RESPONSE:

• **(a) Data Abstraction & Drone Navigation:**
- The data abstraction is the list `flightAltitudes`, storing ordered integer altitudes representing waypoint heights along the drone's 3D flight path.
- *Necessity:* Flight paths vary in length depending on mission distance. A list allows dynamic waypoint sequences to be validated with a single iterative algorithm regardless of whether a delivery has 4 waypoints or 40 waypoints.

• **(b) Algorithm Verification Logic:**
- *Flag Initialization:* Initializes `isSafe <- true` and counter `violatingWaypoints <- 0`.
- *Envelope Testing:* Loops through each altitude `alt` using `FOR EACH`.
- *Compound Condition:* Checks `IF (alt < minAltitude OR alt > maxAltitude)`. If either boundary is breached, sets `isSafe <- false` and increments `violatingWaypoints`.
- *Output Selection:* If `isSafe` remains `true`, approves takeoff. Otherwise, concatenates and returns a formatted violation message.

• **(c) Execution Trace with [min: 90, max: 150]:**
- 120: 90 <= 120 <= 150 (Safe)
- 145: 90 <= 145 <= 150 (Safe)
- 95: 90 <= 95 <= 150 (Safe)
- 110: 90 <= 110 <= 150 (Safe)
- 80: 80 < 90 -> **Violation!** `isSafe <- false`, `violatingWaypoints` becomes 1.
- 130: 90 <= 130 <= 150 (Safe)
- Final return: **"Safety Violation: Ineligible Waypoints = 1"**.

• **(d) Two Distinct Testing Calls:**
- *Call 1 (Approval Branch):* `verifyFlightSafety(flightAltitudes, 75, 150)`. All altitudes fall between 75m and 150m. Returns `"Flight Path Approved"`.
- *Call 2 (Rejection Branch):* `verifyFlightSafety(flightAltitudes, 100, 140)`. Waypoint 145 exceeds max and waypoints 95 and 80 fall below min. Returns `"Safety Violation: Ineligible Waypoints = 3"`.

OFFICIAL COLLEGE BOARD 9-POINT SCORING RUBRIC:

Criteria / Evaluated Performance Skill	Points Allocated
(a) Identifies `flightAltitudes` list storing altitude values, and explains that routes have variable numbers of waypoints that require dynamic storage.	**2 pts**
(b) Explains algorithm: sets `isSafe = true`, loops through list, flags any altitude outside `[minAltitude, maxAltitude]` envelope, accumulates violation count, and returns approval or violation string.	**3 pts**
(c) Waypoint checks: 120 (safe), 145 (safe), 95 (safe), 110 (safe), 80 (< 90, violation!), 130 (safe). `isSafe` becomes false, `violatingWaypoints = 1`. Returns 'Safety Violation: Ineligible Waypoints = 1'.	**2 pts**
(d) Call 1: `verifyFlightSafety(flightAltitudes, 70, 160)` -> all waypoints within bounds, returns 'Flight Path Approved'. Call 2: `verifyFlightSafety(flightAltitudes, 100, 140)` -> multiple violations, returns safety violation string.	**2 pts**
TOTAL SCORING MAXIMUM	**9 / 9 Points**

Question 95 · Model Solution & Scoring Rubric

Mobile Quiz Battle Streak & Scoring Engine ❖ Total Available Points: 9 Points

COMPREHENSIVE STEP-BY-STEP RESPONSE:

• **(a) Boolean List Data Abstraction:**
- The data abstraction is the list `matchAnswers`, which stores ordered Boolean primitives (`true`/`false`).
- *Suitability:* Quiz answers are binary (either right or wrong). A Boolean list compactly represents this stream of outcomes while preserving exact sequential question order necessary for streak calculations.

• **(b) Streak Scoring Algorithm:**
- *State Tracking:* Tracks `totalScore <- 0` and consecutive streak `currentStreak <- 0`.
- *Correct Answer Branch:* When `isCorrect = true`, increments `currentStreak` by 1 and adds `pointsPerQuestion` to `totalScore`. Checks `IF (currentStreak >= 3)` and awards an extra `streakBonus` for sustained performance.
- *Incorrect Answer Branch:* When `isCorrect = false`, the `ELSE` branch resets `currentStreak <- 0`, breaking the multiplier chain.

• **(c) Step-by-Step Execution Trace (points = 100, bonus = 50):**
- Q1 (`true`): streak = 1; score = 0 + 100 = **100**.
- Q2 (`true`): streak = 2; score = 100 + 100 = **200**.
- Q3 (`true`): streak = 3 (streak >= 3!); score = 200 + 100 + 50 = **350**.
- Q4 (`false`): streak resets to 0; score remains **350**.
- Q5 (`true`): streak = 1; score = 350 + 100 = **450**.
- Q6 (`true`): streak = 2; score = 450 + 100 = **550**.
- Returned total score: **550**.

• **(d) Compounding Streak Effect:**
- If Q4 were `true`, the streak would continue unbroken: Q4 (streak 4 -> 150 pts), Q5 (streak 5 -> 150 pts), Q6 (streak 6 -> 150 pts).
- Total score would become 350 + 150 + 150 + 150 = 800.
- An unbroken streak compounds score growth because every subsequent correct question qualifies for the streak bonus.

OFFICIAL COLLEGE BOARD 9-POINT SCORING RUBRIC:

Criteria / Evaluated Performance Skill	Points Allocated
(a) Identifies `matchAnswers` storing Boolean values (`true`/`false`), and explains that Booleans directly capture binary correctness without excess memory.	**2 pts**
(b) Explains algorithm: iterates through answers, increments streak and adds base points on `true`, checks if streak >= 3 to award bonus, and resets streak to 0 on `false`.	**3 pts**
(c) Traces trace correctly: Q1 (streak 1, score 100), Q2 (streak 2, score 200), Q3 (streak 3, score 350 [bonus]), Q4 (streak 0, score 350), Q5 (streak 1, score 450), Q6 (streak 2, score 550). Returns 550.	**2 pts**
(d) Explains compounding effect: if Q4 is true, streak is not broken. Q4 gives 150 (score 500), Q5 gives 150 (score 650), Q6 gives 150 (score 800), adding 250 points total.	**2 pts**
TOTAL SCORING MAXIMUM	**9 / 9 Points**

Question 96 · Model Solution & Scoring Rubric

Hospital Emergency Room Patient Triage Priority Queue ❖ Total Available Points: 9 Points

COMPREHENSIVE STEP-BY-STEP RESPONSE:

• **(a) Data Abstraction & Queue Order:**
- The data abstraction is `patientAcuity`, a list of integer acuity scores ranging from 1 to 5.
- *Order Representation:* The list indices (1, 2, 3...) reflect patient arrival timestamps, allowing the hospital system to identify both clinical severity and arrival sequence.

• **(b) Linear Search Algorithm Implementation:**
- *Initialization:* Creates an empty results list `matchingPatientIndices <- []` and index counter `i <- 1`.
- *Sequencing & Iteration:* Loops `REPEAT LENGTH(acuityList) TIMES` to visit each patient in order.
- *Selection:* Tests `IF (acuityList[i] = targetAcuity)`. If a match is found, `APPEND(matchingPatientIndices, i)` records the patient's queue position.
- *Index Advance:* `i <- i + 1` advances sequentially to ensure every entry is inspected.
- *Return:* Returns the compiled list of patient indices.

• **(c) Execution with targetAcuity = 1:**
- Patient 1: acuity 3 ≠ 1
- Patient 2: acuity 4 ≠ 1
- Patient 3: acuity 1 = 1 -> **Append 3**
- Patient 4: acuity 2 ≠ 1
- Patient 5: acuity 5 ≠ 1
- Patient 6: acuity 1 = 1 -> **Append 6**
- Patient 7: acuity 3 ≠ 1
- Result: Returns **`[3, 6]`**. Medical staff immediately attend to Patient #3 and Patient #6.

• **(d) Behavior with Non-Existent Target (targetAcuity = 6):**
- Because all triage levels are between 1 and 5, `acuityList[i] = 6` will evaluate to `false` on all 7 iterations.
- The `APPEND` statement will never execute, and the procedure safely returns an empty list: **`[]`** without runtime errors.

OFFICIAL COLLEGE BOARD 9-POINT SCORING RUBRIC:

Criteria / Evaluated Performance Skill	Points Allocated
(a) Identifies `patientAcuity` list storing integer triage levels 1–5, explaining that patient arrival order is preserved by index position.	**2 pts**
(b) Explains algorithm: initializes empty `matchingPatientIndices` list, iterates index `i` from 1 to list length, compares `acuityList[i]` to `targetAcuity`, and appends matching index `i`.	**3 pts**
(c) Traces execution: checks indices 1–7. Acuity 1 occurs at index 3 and index 6. Returns `[3, 6]` (patients #3 and #6).	**2 pts**
(d) Explains that the condition `acuityList[i] = 6` is never true, so no items are appended, and the procedure cleanly returns an empty list `[]`.	**2 pts**
TOTAL SCORING MAXIMUM	**9 / 9 Points**

Question 97 · Model Solution & Scoring Rubric

Smart Home Energy Conservation & Peak-Hours Load Shedding ❖ Total Available Points: 9 Points

COMPREHENSIVE STEP-BY-STEP RESPONSE:

• **(a) 24-Hour List Abstraction:**
- The data abstraction is `hourlyUsage`, containing 24 decimal values representing energy consumption.
- *Index Mapping:* In AP CSP 1-based indexing, index 1 represents Hour 1 (1:00 AM), index 12 represents 12:00 PM, and index 24 represents 12:00 Midnight, creating an intuitive 1-to-1 temporal mapping.

• **(b) Algorithmic Logic & Targeted Window Iteration:**
- *Window Bound:* Sets loop index `i <- peakStartHour`.
- *Loop Condition:* The `REPEAT UNTIL (i > peakEndHour)` condition inspects ONLY the hours within the peak utility window without iterating over irrelevant morning or night hours.
- *Accumulation & Violation Counting:* Adds `hourlyDraw` to `peakTotal` and checks `IF (hourlyDraw > limitKWh)` to increment `hoursOverLimit`.
- *Shedding Decision:* If 2 or more peak hours exceed the threshold, triggers `triggerLoadShedding()` and alerts the user.

• **(c) Execution Trace for Hours 16–19 (limit = 5.0):**
- Index 16: 3.8 kWh <= 5.0 (Normal)
- Index 17: 5.2 kWh > 5.0 -> **Violation 1**
- Index 18: 6.1 kWh > 5.0 -> **Violation 2**
- Index 19: 5.8 kWh > 5.0 -> **Violation 3**
- `hoursOverLimit` = 3.
- Condition `3 >= 2` evaluates to `true`.
- Return value: **"High Demand: Load Shedding Activated"**.

• **(d) Impact of Increasing Limit to 6.5 kWh:**
- If `limitKWh` is raised to 6.5, all 4 peak readings (3.8, 5.2, 6.1, 5.8) are less than 6.5.
- `hoursOverLimit` remains 0.
- The check `0 >= 2` evaluates to `false`, the load shedding trigger is bypassed, and the procedure returns: **"Normal Demand: Grid Balanced"**.

OFFICIAL COLLEGE BOARD 9-POINT SCORING RUBRIC:

Criteria / Evaluated Performance Skill	Points Allocated
(a) Identifies `hourlyUsage` list storing 24 decimal power values, and explains that indices 1 through 24 correspond directly to hours 1:00 AM through 12:00 Midnight.	**2 pts**
(b) Explains algorithm: sets counter `i = peakStartHour`, loops until `i > peakEndHour`, sums peak power, counts hours exceeding `limitKWh`, and triggers load shedding if violations >= 2.	**3 pts**
(c) Evaluates hours 16–19: Hr 16 (3.8 <= 5.0), Hr 17 (5.2 > 5.0 -> violation 1), Hr 18 (6.1 > 5.0 -> violation 2), Hr 19 (5.8 > 5.0 -> violation 3). `hoursOverLimit = 3 >= 2`. Returns 'High Demand: Load Shedding Activated'.	**2 pts**
(d) With limit 6.5: Hr 17 (5.2), Hr 18 (6.1), Hr 19 (5.8) are all <= 6.5. `hoursOverLimit = 0`. Condition `0 >= 2` is false. Returns 'Normal Demand: Grid Balanced'.	**2 pts**
TOTAL SCORING MAXIMUM	**9 / 9 Points**

Question 98 · Model Solution & Scoring Rubric

E-Commerce Shopping Cart Discount & Free Shipping Calculator ❖ Total Available Points: 9 Points

COMPREHENSIVE STEP-BY-STEP RESPONSE:

• **(a) Data Abstraction & Dynamic Cart Sizes:**
- The data abstraction is `cartItems`, a list of floating-point numbers representing individual product purchase prices.
- *Dynamic Capacity:* A customer may purchase 1 item or 50 items. A list dynamically accommodates any number of entries, allowing a single `FOR EACH` loop to compute the subtotal without changing code.

• **(b) Sequential Algorithm Implementation:**
- *Step 1 (Summation):* Iterates through `itemsList` adding each `price` to `rawSubtotal`.
- *Step 2 (Promotional Discount):* Evaluates `IF (promoCode = "SAVE20")`. If valid, reduces subtotal by 20% (`* 0.80`).
- *Step 3 (Shipping Eligibility):* Default shipping is $10.00. Evaluates compound Boolean check `(isPrimeMember = true OR discountedSubtotal >= 100.00)`. If either condition is met, sets shipping to $0.00.
- *Step 4 (Final Total):* Adds `discountedSubtotal + shippingFee` and returns the final charge.

• **(c) Execution with promoCode = "SAVE20", isPrimeMember = false:**
- Raw Subtotal: 25.00 + 15.50 + 42.00 + 8.50 + 60.00 = **$151.00**.
- Discount: 151.00 × 0.80 = **$120.80**.
- Shipping Check: `(false = true OR 120.80 >= 100.00)` -> `false OR true` = `true`. Shipping fee is **$0.00**.
- Final Total: 120.80 + 0.00 = **120.80**.

• **(d) Two Distinct Testing Calls:**
- *Call 1 (Free Shipping Branch):* `computeOrderTotal([20.00], "NONE", true)`. `isPrimeMember = true` triggers free shipping ($0.00). Total is $20.00.
- *Call 2 (Paid Shipping Branch):* `computeOrderTotal([20.00], "NONE", false)`. Not prime, and subtotal $20.00 < $100. Shipping remains $10.00. Total is $30.00.

OFFICIAL COLLEGE BOARD 9-POINT SCORING RUBRIC:

Criteria / Evaluated Performance Skill	Points Allocated
(a) Identifies `cartItems` list storing price decimals, and explains that customer shopping carts contain variable quantities of items requiring dynamic list storage.	**2 pts**
(b) Explains algorithm: iterates with `FOR EACH` to sum prices, checks promo code to apply 20% discount, applies compound condition `(isPrimeMember = true OR discountedSubtotal >= 100)` for free shipping, and returns total.	**3 pts**
(c) Calculates: rawSubtotal = 25.00 + 15.50 + 42.00 + 8.50 + 60.00 = 151.00. Discount 20%: 151.00 × 0.80 = 120.80. Shipping: 120.80 >= 100 -> shipping = 0. Final total = 120.80.	**2 pts**
(d) Call 1: `computeOrderTotal(cartItems, "NONE", false)` -> subtotal 151.00 >= 100 -> shipping $0. Call 2: `computeOrderTotal([15.00, 20.00], "NONE", false)` -> subtotal 35.00 < 100 and not prime -> shipping $10.00.	**2 pts**
TOTAL SCORING MAXIMUM	**9 / 9 Points**

Question 99 · Model Solution & Scoring Rubric

Cybersecurity Network Traffic Anomaly & DDoS Detection ❖ Total Available Points: 9 Points

COMPREHENSIVE STEP-BY-STEP RESPONSE:

• **(a) Time-Series List Data Abstraction:**
- The data abstraction is `trafficRates`, an ordered list of integer throughput measurements (packets/second) sampled every second.
- *Anomaly Detection:* Storing samples in sequential list order allows algorithms to inspect trends over time, distinguishing brief transient spikes from sustained malicious flooding attacks.

• **(b) Algorithm Implementation Details:**
- *Peak Initialization:* Initializes `maxObserved <- ratesList[1]` and counter `spikeCount <- 0`.
- *Traversal:* Loops through each rate in `ratesList`.
- *Peak Tracking:* `IF (rate > maxObserved)` continuously updates the highest throughput observed.
- *Spike Detection:* `IF (rate >= baselineMax * spikeThreshold)` identifies readings that exceed normal baseline capacity by the specified multiplier.
- *Alert Escalation:* If 3 or more seconds exhibit attack-level traffic, alerts the network administrator with the peak packet volume.

• **(c) Execution Trace with baselineMax = 600, spikeThreshold = 4:**
- Threshold calculation: 600 × 4 = **2,400 packets/sec**.
- Scanning rates:
- 450, 480, 510, 490, 520: all < 2400.
- 2400: matches threshold -> `spikeCount` = 1, `maxObserved` = 2400.
- 3100: exceeds threshold -> `spikeCount` = 2, `maxObserved` = 3100.
- 2800: exceeds threshold -> `spikeCount` = 3.
- 500, 470: < 2400.
- Post-loop evaluation: `spikeCount = 3 >= 3` is `true`.
- Return value: **"CRITICAL ALERT: DDoS Attack Detected. Peak = 3100"**.

• **(d) False Positive Mitigation:**
- Harmless network events (like a user downloading an operating system update) can cause brief 1-second packet spikes.
- Requiring a sustained threshold (`spikeCount >= 3`) filters out momentary benign anomalies, preventing alarm fatigue and false positives while reliably catching true multi-second DDoS floods.

OFFICIAL COLLEGE BOARD 9-POINT SCORING RUBRIC:

Criteria / Evaluated Performance Skill	Points Allocated
(a) Identifies `trafficRates` list storing integer packets-per-second, explaining that time-series sequencing enables detecting sustained spikes over time.	**2 pts**
(b) Explains algorithm: sets `maxObserved = ratesList[1]`, loops with `FOR EACH`, updates peak, evaluates if rate exceeds `baselineMax * spikeThreshold`, and triggers alert if `spikeCount >= 3`.	**3 pts**
(c) Traces trace: baseline 600 × 4 = 2400 threshold. Spike rates: 2400 (match 1), 3100 (match 2), 2800 (match 3) -> `spikeCount = 3`, `maxObserved = 3100`. Returns 'CRITICAL ALERT: DDoS Attack Detected. Peak = 3100'.	**2 pts**
(d) Explains that requiring `spikeCount >= 3` mitigates false positives from harmless momentary network jitter or large file downloads, ensuring alerts fire only for sustained attacks.	**2 pts**
TOTAL SCORING MAXIMUM	**9 / 9 Points**

Question 100 · Model Solution & Scoring Rubric

City Bike-Share Station Balancing & Redistribution Algorithm ❖ Total Available Points: 9 Points

COMPREHENSIVE STEP-BY-STEP RESPONSE:

• **(a) Station Index Data Abstraction:**
- The data abstraction is `stationBikes`, a 1-indexed list of integers representing current available bicycles at each bike station.
- *Index Correspondence:* In AP CSP pseudocode, index 1 corresponds to Station 1, index 2 to Station 2, and so forth, providing a direct mapping between software array positions and physical city locations.

• **(b) Rebalancing Algorithm Logic:**
- *Tracking Setup:* Initializes empty list `emptyStations <- []` and accumulator `totalDeficit <- 0`.
- *Iteration:* Loops `REPEAT LENGTH(bikeList) TIMES` using counter `i`.
- *Deficit Check:* `IF (currentCount < targetCapacity)` calculates the shortfall (`targetCapacity - currentCount`) and adds it to `totalDeficit`.
- *Critical Station Identification:* Nested condition `IF (currentCount <= 3)` identifies stations near total exhaustion and adds their index to `emptyStations`.
- *Dispatch Decision:* If 2 or more stations are critical (`LENGTH(emptyStations) >= 2`), dispatches a redistribution van and returns a status confirmation.

• **(c) Execution Trace with targetCapacity = 10:**
- Stn 1: 18 bikes >= 10 (Surplus).
- Stn 2: 3 bikes < 10 -> deficit = 10 - 3 = 7. `3 <= 3` is true! **Append 2 to emptyStations**.
- Stn 3: 22 bikes >= 10 (Surplus).
- Stn 4: 1 bike < 10 -> deficit = 10 - 1 = 9. `1 <= 3` is true! **Append 4 to emptyStations**.
- Stn 5: 14 bikes >= 10 (Surplus).
- Final values:
- `emptyStations` = **`[2, 4]`** (Stations 2 and 4).
- `totalDeficit` = 7 + 9 = **16 bicycles**.
- `LENGTH(emptyStations) = 2 >= 2` evaluates to `true`.
- Return string: **"Van Dispatched: Critical Stations = 2"**.

• **(d) Impact of Adding Bikes to Station 4:**
- If Station 4 has 5 bikes: 5 is still < 10 (shortfall = 5, totalDeficit = 12), BUT `5 <= 3` evaluates to `false`.
- Therefore, Station 4 is NOT appended to `emptyStations`.
- `emptyStations` would contain only `[2]` (`LENGTH = 1`).
- The dispatch check `1 >= 2` evaluates to `false`, cancelling the van run and returning: **"Stations Balanced: No Van Required"**.

OFFICIAL COLLEGE BOARD 9-POINT SCORING RUBRIC:

Criteria / Evaluated Performance Skill	Points Allocated
(a) Identifies `stationBikes` list storing bicycle counts, explaining that 1-based indices 1 through 5 correspond directly to physical Stations 1 through 5.	**2 pts**
(b) Explains algorithm: iterates with index `i`, checks if bikes < `targetCapacity`, accumulates shortfall into `totalDeficit`, checks critical threshold <= 3 to append index to `emptyStations`, and dispatches van if critical stations >= 2.	**3 pts**
(c) Traces trace: Stn 1 (18 > 10), Stn 2 (3 < 10, deficit 7, <= 3 -> append 2), Stn 3 (22 > 10), Stn 4 (1 < 10, deficit 9, <= 3 -> append 4), Stn 5 (14 > 10). `emptyStations = [2, 4]`, `totalDeficit = 16`. Returns 'Van Dispatched: Critical Stations = 2'.	**2 pts**
(d) Explains: If Stn 4 has 5 bikes, count 5 is > 3, so Stn 4 is not added to `emptyStations`. Only Stn 2 remains in list (`LENGTH = 1 < 2`). Van is not dispatched; returns 'Stations Balanced: No Van Required'.	**2 pts**
TOTAL SCORING MAXIMUM	**9 / 9 Points**

★ ❖ ★

Thank You, Future Software Engineers & Computer Scientists!

YOU HAVE COMPLETED THE 100-QUESTION UNIT 3 MASTER EXAMINATION

Congratulations on rigorously working through all 100 questions of **Unit 3: Algorithms & Programming**. From tracing complex nested conditionals and 1-indexed list operations, to navigating robot grid obstacles and designing Create Performance Task procedures, you have practiced the most heavily weighted domain on the entire AP Computer Science Principles examination.

Algorithms are the core computational engine of modern civilization. By mastering procedural abstraction, data structures, and algorithmic efficiency, you possess the analytical mindset to build scalable software and excel on exam day.

"Programming is not just about what you know; it is about what you can figure out through logic, abstraction, and perseverance."